

Работа со скриптами системы БАЗИС

Руководство пользователя

10 марта 2021 г.

Информация, содержащаяся в данном документе, может быть изменена
без предварительного уведомления.

Никакая часть данного документа не может быть воспроизведена или передана
в любой форме и любыми способами в каких-либо целях
без письменного разрешения ООО БАЗИС-Центр.

©2021 ООО БАЗИС-Центр. С сохранением всех прав.

БАЗИС, БАЗИС-Мебельщик, логотип БАЗИС
являются зарегистрированными торговыми марками ООО БАЗИС-Центр.

Остальные упомянутые в документе торговые марки
являются собственностью их законных владельцев.

Содержание

Общие сведения 19

Демонстрационные примеры БАЗИС-скрипт 19

Редактор скриптов 19

Запуск редактора 19

Интерфейс Редактора скриптов 19

Команды редактора 19

Вкладка скрипта 20

Отладчик скриптов 20

Организация взаимодействия с пользователем 20

Элементы управления на панелях инструментов 20

Элементы управления вспомогательной панели Свойства 21

Выдача сообщений пользователю 21

Синтаксис языка 21

Список ключевых слов языка программирования БАЗИС-Script: 22

Список зарезервированных слов: 22

Комментарии 23

Переменные 23

Константы 24

Арифметические операции 25

Логические операции 25

Операции сравнения 26

Оператор if...else 26

Оператор switch 27

Оператор условного выражения 28

Операторы break и continue 28

Цикл for 29

Цикл while 29

Цикл do...while 30

Оператор with 30

Оператор try...catch...throw 31

Функции 31

Встроенные объекты скриптового языка 32

Общие сведения 32

Объект Global 33

Объект Number 33
Объект Boolean 33
Объект String и операции над строками 33
Объект RegExp (регулярные выражения) 34
Объект Array и функции работы с массивами 36
Объект Date 39
Объект Math 40
Модуль числа 40
Округление чисел 40
Определение максимума и минимума 41
Возведение в степень 41
Вычисление квадратного корня 41
Генератор случайных чисел 41
Методы тригонометрии 41
Вычисление натурального логарифма 42
Общее для всех модулей 42
Системные объекты 42
NewVector 42
NewPoint 43
NewContour 43
NewCOMObject 43
NewForm 44
Глобальные константы 44
AxisX 44
AxisY 44
AxisZ 44
Axis_X 44
Axis_Y 44
Axis_Z 45
Глобальные перечисляемые типы 45
TProjection3dEnum 45
PanelSideEnum 45
SalonAnimationType 45
TextureDir 46
WindowPosition 46
FurnSchemeType 46
SchemeFurnPos 46

ErrorType ... **47**
DatumMode ... **47**
FurniturePosition ... **48**
ElementType ... **48**
FurnPositionMode ... **48**
Глобальные свойства и методы ... **48**
Глобальные свойства ... **48**
system ... 48
Model ... 49
Action ... 49
ActiveMaterial ... 49
AnimationType ... 49
Глобальные методы ... **49**
prompt ... 49
alert ... 50
confirm ... 50
OpenFurniture ... 50
SelectAll ... 51
UnSelectAll ... 51
ViewAll ... 51
SetCamera ... 51
GetPoint ... 52
GetObject ... 52
GetPanel ... 52
GetEdge ... 52
AddPanel ... 53
AddFrontPanel ... 53
AddHorizPanel ... 54
AddVertPanel ... 54
AddExtrusion ... 55
AddTrajectory ... 55
AddBlock ... 56
AddAssembly ... 56
AddDraftBlock ... 56
AddCopy ... 57
DeleteNewObjects ... 57
DeleteObject ... 57
StartEditing ... 58
BeginBlock ... 58
EndBlock ... 58
NewButtonInput ... 58
NewFloatInput ... 59
NewNumberInput ... 59
NewMaterialInput ... 60
NewButtMaterialInput ... 60

NewFurnitureInput 61
FormatMatName 61
OrientCamera 61
ExtractMatName 61
ExtractMatCode 62
Geometry2D 62
Методы 62
Intersect 62
Compare 62
Distance 63
Area 63
System 63
Методы 64
include 64
log 64
fileExists 64
writeTextFile 64
askWriteTextFile 65
readTextFile 65
askReadTextFile 65
secureExec 65
exec 66
getFileName 67
getFileNameWithoutExtension 67
askFileNameSave 67
askFolder 67
Свойства 68
apiVersion 68
developerApiVersion 68
Model3D 69
Свойства 69
Selected 69
SelectionCount 69
Selections 69
Count 69
Objects 70
Action3D 70
Свойства 71
Interactive 71
MouseX 71
MouseY 72
Pos3 72
ViewDir 72
UpDir 72

RightDir72
ShowPoints73
ShowEdges73
Hint73
ErrorHint73
BlinkHint73
OnClick73
OnMove74
OnStart74
OnFinish74
Properties74
OnRayTraceFinished75
ModelFilename75

Методы75

Continue75
Commit75
Finish75
Cancel76
AsyncExec76
BeginOrtho376
EndOrtho377
CursorToClosestPoint77
CursorToClosestLine77
CursorToMiddleOfLine77
Find3DPoint77
Find3DPointXZPlane77
Get3Dobject77
RayTraceScene78
LoadModel78
SaveModel78
NewModel78
NewFurniture78
NewFragment78
Revert79
ArrangePositions79
ReplaceFurniture79
ChooseMaterial79

ScriptProperty 79

Свойства 81

Name 81
Enabled 81
ChildrenEnabled 81
Visible 81
OnChange 81
Count 82
Items 82
Expanded 82
OnDeactivate 82
OnValueValidate 82
ValueValid 83
NameEditable 83
BackColor 83
PopupMenu 83
DropDownMenu 83
OnActivate 83
OnValueChange 84
Tag 84

Методы 84

Clear 84
NewCOMObject 84
Save 84
Load 85
NewGroup 85
NewImage 85
NewString 86
NewBool 86
NewNumber 86
NewButton 87
NewMaterial 87
NewButt 87
NewFurniture 88
NewColor 88
NewSeparator 88
Validate 88

ScriptGroupProperty 89

Свойства 89

Image 89
MaxHeight 89
Scrollable 89

ScriptStringProperty 90

Свойства 90

Value 90

ScriptBooleanProperty 90

Свойства 90

Value 90

ScriptNumberProperty 90

Свойства 91

MaxValue 91

MinValue 91

ValueStep 91

Value 91

ScriptButtonProperty 92

Свойства 92

Value 92

OnClick 92

ScriptMaterialProperty 92

Свойства 92

Width 92

Методы 93

SetActive 93

ScriptButtProperty 93

Свойства 93

Thickness 93

Width 93

Методы 93

SetActive 93

ScriptFurnitureProperty 94

Свойства: 94

Value 94

ScriptColorProperty 94

Свойства 94

Value 94

ScriptMenu 94

Свойства 95

Name 95

OnChange 95

Count 95

Ite ms95

Store 95

Методы 96

Clear 96

Save 96

Load 96
NewGroup 96
NewString 97
New Bool97
NewNumber 97
NewButton 97
ScriptForm 97
Свойства 98
Properties 98
Caption 98
Width 99
Height 99
MinWidth 99
MinHeight 99
Visible 99
Left 100
Top 100
OKButton 100
OKButtonCaption 100
CancelButton 100
CancelButtonCaption 100
OnClose 101
OnShow 101
Resizable 101
Dockable 101
OnOkButtonClick 101
OnCancelButtonClick 102
Методы 102
Show 102
ShowModal 102
Close 102
FurnMaterial 102
Свойства 103
Name 103
Thickness 103
Width 103
Методы 103
Make 103
Vector 103
Свойства: 104
x 104
y 104
z 104

Point 104
Свойства: 104
x 104
y 105
Edge3 105
Свойства 105
First 105
Last 105
GFirst 105
GLast 105
Object3 106
Свойства: 107
Name 107
ArtPos 107
Owner 108
Visible 108
Selected 108
List 108
AsList 108
AsPanel 109
Position 109
PositionX 109
PositionY 109
PositionZ 109
GMax 109
GMin 110
GSize 110
GabMin 110
GabMax 110
UserPropCount 111
UserProperty 111
UserPropertyName 111
Методы 111
SetDefaultTransform 111
Translate 112
Rotate 112
TranslateGCS 112
RotateGCS 112
RotateX 113
RotateY 113
RotateZ 113
Orient 113
OrientGCS 114
Reflect 115
ToObject 115

ToGlobal . . . 115
NToObject . . . 116
NToGlobal . . . 116
Build . . . 116
IsOwner . . . 116
List3D . . . 116
Свойства: . . . 117
Count . . . 117
IsElastic . . . 117
Objects . . . 117
FurnType . . . 118
SalonType . . . 118
Методы: . . . 118
ElasticResize . . . 118
Load . . . 118
Panel . . . 119
Свойства . . . 119
Contour . . . 119
Butts . . . 120
Cuts . . . 120
Plastics . . . 120
ContourHeight . . . 120
ContourWidth . . . 120
Thickness . . . 121
MaterialName . . . 121
MaterialWidth . . . 121
Методы . . . 122
AddButt . . . 122
AddCut . . . 123
AddPlastic . . . 123
IsButtVisible . . . 123
Extrusion . . . 124
Свойства . . . 124
Contour . . . 124
Thickness . . . 124
MaterialName . . . 124
Trajectory . . . 125
Свойства . . . 125
Contour2D . . . 125
Trajectory2D . . . 125
MaterialName . . . 126
Пример: . . . 126

Block 126

Свойства: 126

IsFastener 126

Assembly 127

Свойства 127

AnimType 127

PanelButts 127

Свойства 127

Count 127

Методы 128

Butts[] 128

Add 128

PanelButt 128

Свойства 129

ElemIndex 129

Sign 129

Material 129

Thickness 129

Width 130

ClipPanel 130

Overhung 130

Allowance 130

CutIndex 130

Profile 131

PanelPlastics 131

Свойства 131

Count 131

Методы 132

Plastics[] 132

Add 132

PanelPlastic 132

Свойства: 132

Material 132

Thickness 133

TextureOrientation 133

PanelCuts 133

Свойства 133

Count 133

Методы 134

Add 134

Cuts 134

PanelCut 134

Свойства 134

Name 134
Sign 134
Trajectory 135
Contour 135

Contour2D 135

Свойства 137

Count 137
Objects 137
Width 138
Height 138
Min 138
Max 138

Методы 138

Clear 138
Move 139
Rotate 139
AddRectangle 139
AddRoundRect 139
AddLine 140
AddCircle 140
AddArc 140
AddArc3 140
AddEquidistantRecursive 141
Subtraction 141
Addition 141
RoundingEx 141
FacetEx 142
Rounding 142
Facet 142
Find 143
Fit 143
Elastic 143
Symmetry 143
Load 144
OrderContours 144
AddEquidistant 144
IsPointInside 144
IsInContour 145
IIsClosedContour 145
IsContourRectangle 145

InControl 145

Свойства 145

id 145

Enabled . . . 146
Visible . . . 146
Hint . . . 146
OnChange . . . 146
InButton . . . 146
Свойства: . . . 147
Caption . . . 147
Combo . . . 147
Методы: . . . 147
NewSubMenu . . . 147
InFloat . . . 147
Свойства . . . 147
Value . . . 147
ReadOnly . . . 148
Fixed . . . 148
InNumber . . . 148
Свойства . . . 148
Value . . . 148
ReadOnly . . . 148
Fixed . . . 149
InMaterial . . . 149
Свойства . . . 149
Name . . . 149
Thickness . . . 149
Width . . . 149
Методы . . . 150
SetActive . . . 150
Apply . . . 150
InButtMaterial . . . 150
Свойства . . . 150
Name . . . 150
Sign . . . 151
Thickness . . . 151
Width . . . 151
Overhung . . . 151
Allowance . . . 151
ClipPanel . . . 151
InFurniture . . . 152
Свойства . . . 152
DatumModeFilter . . . 152
DatumMode . . . 152
Методы . . . 152

Mount . . . 152
Mount1 . . . 153
MountScheme . . . 153
MountBox . . . 153
ModelInspector . . . 153
Свойства . . . 154
ErrorList . . . 154
Options . . . 154
Методы . . . 154
Run . . . 154
InspectorOptions . . . 154
Свойства . . . 155
ObjIntersectionAnalyze . . . 155
FastIntersectionAnalyze . . . 155
FastIncorrectAnalyze . . . 155
PanelNotFixedAnalyze . . . 155
PanelTooLargeAnalyze . . . 155
PlasticTooLargeAnalyze . . . 156
MatNotExistsAnalyze . . . 156
MatOutOfStockAnalyze . . . 156
InspectorError . . . 156
Свойства . . . 156
ErrorType . . . 156
ErrorObjectsCount . . . 157
ErrorObjects . . . 157
ErrorMessage . . . 157
ObjectsNames . . . 157
ImportExportSVG . . . 157
Свойства . . . 158
CurveQuality . . . 158
GroupElems . . . 158
Методы . . . 158
Save . . . 158
Load . . . 158
ImportExport . . . 159
Свойства . . . 159
SVG . . . 159
InfFurniture . . . 159
Свойства . . . 159
DatumModeFilter . . . 159
DatumMode . . . 160
Методы . . . 160

Mount 160
Mount1 160
MountScheme 160
MountBox 161
Make 161
EncodeToString 161
DecodeFromString 161
Choose 162
RootProperties 162
Методы 162
NewFurnitureValue 162
Elem2D 162
Свойства 163
ElType 163
Методы 163
IsLine 163
AsLine 163
IsCircle 163
AsCircle 163
IsEllipse 164
AsEllipse 164
IsArc 164
AsArc 164
IsList 164
AsList 165
ObjLength 165
Line2D 165
Свойства 165
Pos1 165
Pos2 165
Arc2D 165
Свойства 166
Pos1 166
Pos2 166
Center 166
ArcDir 166
Circle2D 167
Свойства 167
Center 167
CirRadius 167
Dir 167

Ellipse2D 167
Свойства 168
Center 168
MajorRadius 168
MinorRadius 168
MajorAxisAngle 168
Dir 168
FurnArticle 169
Свойства 169
Name 169
OrderCode 169
OrderName 169
DatumMode 170
Методы 170
NameWithCode 170
БАЗИС-Мебельщик 170
Глобальные константы БАЗИС-Мебельщик 170
p3dFront 170
p3dLeft 170
p3dRight 170
p3dTop 170
p3dBottom 171
p3dIsometric 171
БАЗИС-Смета 171
Глобальные свойства БАЗИС-Смета 171
Panel 171
Holes 171
Material 171
Result 171
Objects 172
EstimateObject 172
Методы 172
AsPanel 172
IsPanel 172
AsButt 173
IsButt 173
AsPlastic 173
IsPlastic 173
AsExtrusionBody 173
IsExtrusionBody 174

AsTrajectoryBody 174
IsTrajectoryBody 174
EstimateObjectList 174
Свойства 174
Count 174
Items 174
PanelHoles 175
Свойства 175
Count 175
Holes 175
PanelHole 175
Свойства 175
Depth 175
Diameter 176
Мастер построения дверей 176
DoorsMaker 176
Методы 176
Silent 176
ShowErrors 176
Save 176
Load 177
Setup 177
Мастер построения ящиков 177
BoxesMaker 177
Методы 177
ShowErrors 177
Save 178
Load 178
Setup 178
БАЗИС-Салон 178
TovarItems 178
Свойства 179
TovarName 179
TovarModel 179
Price 179
IsNotStandart 179
KodTovID 179
Metka 180
Discount 180
Markup 180
Методы 180

FindByName 180
TScItemTovar 181
Свойства 181
Article 181
Name 181
Count 181
Material 182
Price 182
GroupMaterial 182
TypeElement 182
ObjList 182
SalonUtils 183
Свойства 183
PathAttachments 183
SalonId 183
Методы 183
GetFullPathAttachment 183
UserProperties 184
Свойства 184
Value 184
Name 184
Count 184
Методы 185
Remove 185

Приложение I. Создание экранной формы в визуальном режиме 187

Общие сведения

БАЗИС-скрипты представляют собой ориентированные на прикладного программиста инструментальные средства автоматизации проектирования мебели на базе модуля БАЗИС-Мебельщик. БАЗИС-скрипты имеют формат Java-скриптов и сохраняются в файлах с расширением *js*.

Демонстрационные примеры БАЗИС–скрипт

В комплект поставки системы БАЗИС включены демонстрационные примеры иллюстрирующие применение скриптов. Используя эти примеры, можно создавать собственные скрипты.

При установке модуля БАЗИС-Мебельщик файлы скриптов автоматически сохраняются в папку *C:\Users\<CurrentUserName>\Documents\BazisX\Scripts*, где *X* представляет собой номер версии системы БАЗИС.

Редактор скриптов

Запуск редактора

Команды работы со скриптами расположены в разделе **Скрипты** Главного меню модуля БАЗИС-Мебельщик. Команда **Редактор скриптов** позволяет открыть окно редактора. Другие команды раздела совпадают с именами файлов скриптов. Если щелкнуть левой кнопкой мыши по имени скрипта, он автоматически запустится на выполнение. Если выполнить такой щелчок, удерживая нажатой клавишу *<Ctrl>*, будет раскрыто окно **Редактор скриптов**, в котором будет показан текст скрипта (рис. 1).

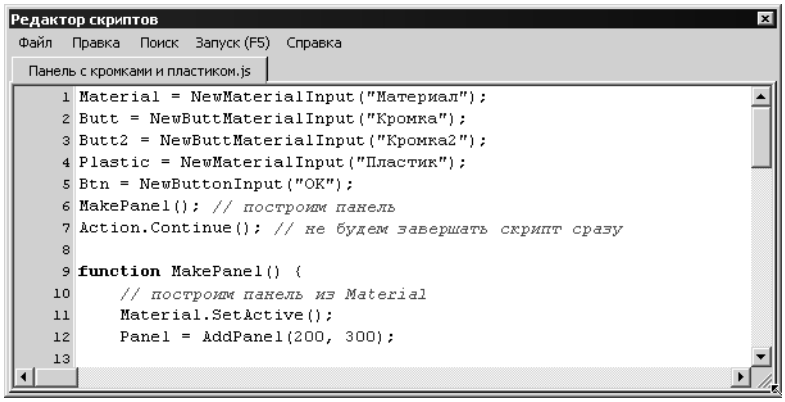


Рис. 1.

Интерфейс Редактора скриптов

Команды редактора

Команды главного меню являются стандартными и позволяют выполнять операции с файлами скриптов, а также закрыть окно редактора. Несколько скриптов могут быть открыты в редакторе одновременно. Их тексты располагаются на вкладках окна. Одна из вкладок является активной. Именно к скрипту, текст

которого находится на этой вкладке, применяются команды редактора. Команда **Запуск** позволяет активный скрипт на выполнение.

Вкладка скрипта

Редактор скриптов представляет собой текстовый редактор. Синтаксис команд скрипта подсвечивается.



Текст скрипта можно подготовить в любом стороннем текстовом редакторе.

Отладчик скриптов



Для отладки скриптов следует использовать Visual Studio Code с плагином Debugger for Bazis-Woodworker's scripts. Чтобы установить плагин, запустите Visual Studio Code и откройте раздел **Extensions**. Введите в поле поиска строку *bazissoft.bazis-debug2* и установите найденный плагин.

Организация взаимодействия с пользователем

Взаимодействие пользователя с программой может выполняться с использованием следующих способов:

- элементы управления (кнопки, поля ввода и т.п.), расположенные на панели инструментов,
- элементы управления, расположенные на вспомогательной панели **Свойства**,
- выдача пользователю сообщений, в том числе, требующих его реакции.

Для простых команд, у которых количество параметров невелико, целесообразно располагать элементы управления на панели инструментов. В качестве аналога можно привести стандартные команды системы БАЗИС, например, команды группы **Операции**. При этом для ускорения доступа к ним автоматически назначаются «горячие клавиши»:

- для доступа к кнопкам *<Shift>+1, <Shift>+2* и т.д.
- для доступа к полям ввода *<Alt>+1, <Alt>+2* и т.д.

При создании сложных скриптов, предназначенных для построения параметрических моделей, целесообразно размещать элементы управления на вспомогательной панели **Свойства**. Это позволяет групировать их, в том числе и на нескольких уровнях.

Элементы управления на панелях инструментов

Чтобы сформировать элементы управления на панелях инструментов, следует использовать следующие методы:

- NewButtonInput
- NewFloatInput
- NewNumberInput

- NewMaterialInput
- NewButtMaterialInput
- NewFurnitureInput

Элементы управления вспомогательной панели Свойства

Чтобы использовать элементы управления вспомогательной панели **Свойства**, следует использовать следующие методы и свойства объекта Action.Properties:

NewGroup – Создать вложенную группу свойств.

NewImage – Создать вложенную группу свойств с рисунком**.**

NewString – Создать свойство типа Строка.

NewBool – Создать свойство типа Да/Нет.

NewNumber – Создать свойство типа Число.

NewButton – Создать свойство с кнопкой редактирования.

NewMaterial – Создать свойство типа Материал.

NewButt – Создать свойство типа Материал кромки.

NewFurniture – Создать свойство типа Фурнитура.

NewColor – Создать свойство типа Цвет.

NewSeparator – Создать разделитель.

PopupMenu – Всплывающее меню.

DropDownMenu – Выпадающее меню.

Выдача сообщений пользователю

Выдача пользователю сообщений обеспечивается использованием следующих методов:

alert – Вывести на экран сообщение или значения параметров.

confirm – Вывести на экран диалог подтверждения или отмены.

Hint – Установить строку подсказки.

ErrorHint –Установить сообщение об ошибке.

Синтаксис языка

Синтаксис языка скриптов похож на C++ и Java. Но в любом языке программирования присутствует своя специфика представления ключевых и зарезервированных слов, составляющих ядро для программирования на этом языке. Ключевые слова всегда доступны программисту, но для их использования нужно придерживаться правильного синтаксиса.

Список ключевых слов языка программирования БАЗИС-Script:

- break
- switch
- for
- typeof
- continue
- try
- new
- while
- const
- finally
- case
- this
- return
- throw
- in
- default
- var
- else
- if
- catch
- do
- function
- with

Список зарезервированных слов:

- boolean
- double
- goto
- interface
- byte
- enum
- implements
- long

- **char**
- **export**
- **import**
- **native**
- **throws**
- **float**
- **int**
- **short**

Комментарии

К комментариям относится часть кода скрипта, которая никогда не будет выполняться программой, однако она очень полезна при дальнейшей разработке. Например, используя комментарии, легко можно найти участок созданной ранее программы или предоставить возможность другим пользователям сделать это. Комментарии могут начинаться с символов <//>, либо вы можете закомментировать участок кода, начав с символа открытия комментария </*> и закрыв, соответственно, символом <*/>.

Примеры:

box(0,0,0,100,100,100,"box1"); //построение параллелепипеда. Все что написано после знака <//> и есть комментарий к коду

/

Этот участок кода никогда не будет исполнен, он нужен лишь для того, чтобы в дальнейшем понять, что делалось в данном скрипте, а может быть для отладки участка программы...

**/*

Весь участок кода начиная с символа </*> и заканчивая <*/> игнорируется программой и является комментарием

Переменные

Для создания переменной используется ключевое слово **var**.

Примеры:

var x; //создаю переменную x без инициализации

var x=100; //создаю переменную x, равную 100 целого типа, возможные значения от -2147483648 до 2147483647.

var x,y=100; //создаю сразу несколько переменных

var x = new Array(4); //создаю массив с 4 элементами(x[0],x[1],x[2],x[3])

var x = "Текст для вывода на экран\nНа второй строке - информация\nНа третьей строке..."; //создаю переменную с текстовым содержанием

var x = true; //создаю переменную логического содержания true(истина) или false(ложь)

var x = 1516.2298; //создаю переменную вещественного типа, возможные значения от -1.17549435E-38 до 3.40282347E+38.



Внимание! Для корректной работы скриптов имена переменных должны начинаться со строчной или заглавной латинской буквы или знака подчеркивания; имя не должно содержать никаких специальных символов, например: !, ?,| и т. д.; имя не должно совпадать с ключевыми словами языка. Если вы некорректно зададите имя переменной, то при выполнении скрипта будет выведено сообщение об ошибке и ее описание.

При задании текстовой переменной можно использовать спецсимволы, приведенные в табл. 1.

Табл.1.

Символ	Описание
\b	Возврат на один символ с его удалением
\t	Горизонтальная табуляция
\n	Новая строка
\v	Вертикальная табуляция
\r	Возврат каретки
\"	Двойная кавычка
\'	Одинарная кавычка
\\	Обратная косая черта

Константы

Для создания констант используется ключевое слово **const**. Отличием констант является то, что их значение нельзя изменить. Основным преимуществом таких типов является меньшее количество байт, занятых этими переменными и, как следствие, ускорение выполнения кода при использовании **const** где это возможно.

Примеры:

const message = "Неизменяемый текст"; //создаю константу текстового типа

Арифметические операции

- Сложение. Пример x+y
- Вычитание. Пример x-y
- Умножение. Пример x*y
- Деление. Пример x/y
- Остаток от деления. Пример: x % y

Для простоты использования в языке скриптов можно использовать сокращенные записи этих операций (табл. 2).

Табл.2.

Операция	Описание
x += y	Сокращенная запись x = x + y
x -= y	Сокращенная запись x = x - y
x *= y	Сокращенная запись x = x * y
x /= y	Сокращенная запись x = x / y
x %= y	Сокращенная запись x = x % y
++x	Сокращенная запись x = x + 1. Увеличение на 1 до присваивания
--x	Сокращенная запись x = x - 1. Уменьшение на 1 до присваивания
x++	Сокращенная запись x = x + 1. Увеличение на 1 после присваивания
x--	Сокращенная запись x = x - 1. Уменьшение на 1 после присваивания

Примеры:

var i = 0;

var a = ++i; //В этом примере переменная i будет сначала увеличена на единицу, а затем переменной a будет присвоен результат. Итог: a = 1.

var i = 0;

var a = i++; //В этом примере значение переменной i сначала будет присвоено переменной a и только после увеличено на 1. Итог: a = 0.

Логические операции

- Логическое "И" - &&
- Логическое "ИЛИ" - ||

Примеры:

var i = 0;

var f = 5;

if (f>i && f!=0) {} //Итог: true, т.к. первое и второе условие - true

var i = 10;

var f = 15;

if (i==f || f==0) {} //Итог: false, т.к. ни первое, ни второе условие неверно.

Операции сравнения

Такие операции необходимы для сравнения одной переменной или константы с другой. В табл. 3 приведен список всех возможных операций сравнения:

Табл.3.

Операция	Описание
x == y	Возвращает true, если x и y равны между собой
x != y	Возвращает true, если x и y не равны между собой
x === y	Возвращает true, если x и y равны между собой и их типы совпадают
x < y	Возвращает true, если x меньше чем y
x <= y	Возвращает true, если x меньше или равен y
x > y	Возвращает true, если x больше чем y
x >= y	Возвращает true, если x больше или равен y

Примеры:

true==1 // Итог: true (true преобразуется в 1)

true===1 // Итог: false (true не преобразуется в 1)

5 <= "5" // Итог: true (строка "5" преобразуется в число)

5 === "5" // Итог: false (строка "5" не преобразуется в число)

5 > 4 // Итог: true (5 больше 4)

Оператор if...else

Принцип действия следующий: сначала проверяется условие **if** и если оно верно, то выполняется следующий код, а если нет, то выполняется код в записи **else**, если он указан

Примеры:

```
if (10 > 5)
{
//Здесь выполняем код, если условие верно
}
else
{
//Здесь выполняем код, если условие не сработало
}

if (10 < 11)
{
//Здесь выполняем код, если условие верно
} //Оператор else пропущен, следовательно если условие не сработает, то никакой код выполняться не будет
```

Оператор switch

Делается выбор по заданному значению. Эта функцию является заменой операторов **if ... else** и является более удобной, когда требуется перебрать множество значений

Пример:

```
var num = 10;
switch (num)
{
case 1:
  //Выполняем код, если num == 1
  break;
case 2:
  //Выполняем код, если num == 2
  break;
case 100:
  //Выполняем код, если num == 100
  break;
default:
  //Выполняем код, если значение найдено не было
```

```
}
```

Оператор условного выражения

Очень компактная и удобная функция для выбора одного из двух доступных значений. Принцип работы:

результат = (условие) ? значение1(если условие верно) : значение2(если условие не сработало)



На языке C/C++ такой оператор работает медленнее обычного оператора **if ... else**. Однако при написании скриптов вопрос оптимизации не стоит так остро и потому такая форма записи встречается часто.

Пример:

```
var S = (1 > 0) ? 1500 : 1600; //Соответственно в этом выражении S = 1500, так как условие верно
```

Операторы break и continue

- **break** – Команда используется внутри цикла для принудительного прекращения выполнения цикла.
- **continue** – Пропустить невыполненную часть цикла и продолжить выполнения цикла со следующими значениями.

Пример:

```
var S = "Имя_объекта1";
var FindedName = "";
for (var i = 0; i < 10; ++i)
{
  var tested = "Имя_объекта" + i;
   if (tested != S) continue; //Продолжаю поиск со следующими значениями
   else
   {
   FindedName = tested;
  break; //Принудительно прекращаю поиск, потому как имя объекта уже найдено и цикл больше не нужен
  }
}

  if (FindedName != "")
   infomsg("Нашли объект по имени: " + FindedName);
  else
```

warning("Объект не найден");

Цикл for

Представляет собой удобный и гибкий способ использования циклов.

Принцип действия:

for (имя_переменной и ее стартовое значение; максимальное значение переменной; изменение переменной)

{

//Тело цикла

}

Пример:

```
var num = 0;
for (var i = 0; i < 10; ++i)
{
  num += i; //При каждом прохождении цикла значение num увеличивается на величину i
}
history(num); //Поместить результат в историю команд
//Несложно догадаться, что после выполнения такого кода программы num = 45 (0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 = 45).
```

Цикл while

Аналог оператора организации цикла **For**; При использовании **while** нет инициализации переменной и ее изменения.

Принцип действия:

while (условие)

{

//Тело цикла

}

Пример:

```
//Предыдущий пример с оператором for можно записать следующим образом:
var num = 0, i = 0; //Инициализация переменной i в отличии от цикла For происходит заранее
while (i < 10)
{
  num += i; //При каждом прохождении цикла значение num увеличивается на величину i
```

++i; //В отличии от цикла For изменяем значение переменной i в теле цикла

}

history(num); //Результат как и в предыдущем примере num = 45

Цикл do...while

Одна из разновидностей цикла **while**. Разница состоит в том, что независимо от условия **while** цикл выполнится хотя бы один раз.

Принцип действия:

do

{

//Тело цикла

} while (условие)

Пример:

//Предыдущий пример с оператором for можно записать следующим образом:

var num = 0, i = 0; //Инициализация переменной i в отличии от цикла For происходит заранее

do

{

num += i; //При каждом прохождении цикла значение num увеличивается на величину i

++i; //В отличии от цикла For изменяем значение переменной i в теле цикла

} while (i < 10)

history(num); //Результат как и в предыдущем примере num = 45

Оператор with

Необходим при неоднократном обращении к одному и тому же объекту.

Пример:

//Без оператора with:

history(Math.PI);

history(Math.abs(-2));

history(Math.max(4, 10, 7, 6));

//С оператором with:

with(Math)

{

history(PI);
history(abs(-2));
history(max(4, 10, 7, 6));
}

Оператор try...catch...throw

Перехват ошибок выполнения кода программы.

Пример:

try
{
//Участок программы, который может привести к ошибке выполнения
var S = 100;
if (S > 99)
throw ("Значение S слишком большое: " + S); //Сделать выброс
//Здесь можно продолжить выполнение программы
}

catch (...)
{
//Этот код будет выполнен в случае возникновения ошибки
critical(e); //Выводим сообщение об ошибке
}

finally
{
//Код выполняется в любом случае
infomsg("Завершили выполнение программы");
}

Функции

Создание собственных функций в файле скрипта

Принцип действия:

function NameOfFunc(arg1,arg2...argn)
{
//Код функции
return 0; //Возвращаемое значение 0 или любое другое значение

```
}
```

Пример:

```
function helloUser(name)
{
infomsg("Доброго времени суток, "+name)
}

var name = GetPar("Ваше имя");
helloUser(name);
```



Вы можете создавать функцию без параметров, а в случае необходимости передать ей параметры и узнать их значение при помощи массива **arguments** внутри функции.

Пример:

```
function multiply() //Функция перемножения
{
var result = 1;
for (var i = 0; i < arguments.length; ++i)
  result *= arguments[i]; //Перемножаю аргументы между собой

return result;
}

infomsg("Результат перемножения 2,3,5,7: " + multiply(2,3,5,7));
```

Встроенные объекты скриптового языка

Общие сведения

Язык БАЗИС-Скрипт предоставляет серию встроенных объектов:

- глобальный объект **Global**,
- объект **Object**,
- объект **Function**,
- объект **Array**,
- объект **String**,
- объект **Boolean**,
- объект **Number**,

- объект **Math**,
- объект **Date**,
- объект **RegExp**,
- объекты ошибок **EvalError**, **RangeError**, **ReferenceError**, **SyntaxError**, **TypeError** и **URiError**.

Объект Global

Это базовый объект языка сценария, который задает свойства и методы глобального пространства имен. Он является объектом верхнего уровня и не имеет родительского объекта. В нем определены свойства конструкторов для всех используемых в сценарии объектов. Этот объект мы можем расширять любыми новыми функциями и методами.

Объект Number

Объект служит для хранения чисел. Создать его очень просто:

var myVar = 1 (число целого типа)

или

var myVar = 12.1125 (число вещественного типа).

Среди методов этого объекта:

- isNaN(variable); //Возвращает true, если переменная не является числом
- isInfinite(variable); //Возвращает true, если значение переменной больше допустимых пределов объекта Number
- parseFloat(variable); //Возвращает число вещественного типа по значению переменной
- parseInt(variable); //Возвращает число целого типа по значению переменной

Объект Boolean

Объект служит для хранения значений true (истина) и false(ложь). Создать его очень просто:

var myVar = true или var myVar = false.

Объект String и операции над строками

Объект служит для хранения текстовых значений. Создать его очень просто:

var myVar = "текстовое значение".

Операции над строками:

- Функция преобразования строки к нижнему регистру: toLowerCase();
- Функция преобразования строки к верхнему регистру: toUpperCase();
- Замена части строки: replace("предыдущее значение","новое значение");
- Получение символа: charAt(num); где num - номер символа

- Длина строки: length
- Получение части строки: substring(start,nmb); где start - начало считывания строки, nmb - количество символов

Примеры:

```
var myStr = "Here is my test String";
history("Моя строка в нижнем регистре выглядит так: " + myStr.toLowerCase());
history("А в верхнем регистре выглядит так: " + myStr.toUpperCase());
history("Замена слов: " + myStr.replace("test","current"));
history("Значение второго символа: " + myStr.charAt(1));
history("Длина строки: " + myStr.length);
history("Первые четыре буквы строки: " + myStr.substring(0,4));
```

Объект RegExp (регулярные выражения)

Регулярные выражения необходимы для проверки соответствия строк указанному шаблону. В скриптах такие выражения должны задаваться в стиле #C, то есть заключаться в косые / и /; либо при помощи оператора new.

В табл. 3.1 приведены специальные символы регулярных выражений.

Табл. 3.1. Специальные символы регулярных выражений

Символ	Описание
\	Экранирование обычных символов. Позволяет использовать в шаблонах спец. символы.
^	Указывает на начало входных данных.
$	Указывает на конец входных данных.
*****	Повтор от нуля до бесконечности.
+	Повтор от одного до бесконечности.
?	Обозначает, что элемент шаблона может присутствовать либо отсутствовать. Также символ вопроса используется при группировке элементов регулярного выражения.
.	Любой символ, кроме перевода строки: \n \r \u2028 или \u2029.
(x)	Находит и запоминает указанную строку.
(?:x)	Находит указанную строку, но не запоминает ее.
x(?=y)	Находит указанную строку x, при условии, что за x следует y.
x(?!y)	Находит указанную строку x, при условии, что за x не следует y.
x|y	Находит x или y.

Табл. 3.1. Специальные символы регулярных выражений

Символ	Описание
{n}	Точное количество повторов.
{n,}	Количество повторов от n до бесконечности.
{n,m}	Количество повторов от n до m.
[xyz]	Находит указанный набор символов.
[^xyz]	Исключает указанный набор символов из поиска.
[\b]	Находит символ backspace.
\b	Находит границу слов, например пробел.
\B	Исключает границу слоев.
\cX	Где X - буква от A до Z. Обозначает контрольный символ в строке.
\d	Находит любую цифру.
\D	Исключает любую цифру из поиска.
\f,\r,\n	Соответствующие спецсимволы form-feed, line-feed, перевод строки.
\s	Находит любые пробельные символов, в том числе сам проблем, перевод строки и т.п.
\S	Исключает пробельные символы из поиска.
\t	Символ табуляции.
\v	Символ вертикальной табуляции.
\w	Любая буква.
\W	Исключение любой буквы.
\n	Обратная ссылка на n-ю запомненную скобками подстроку.
\0	Символ NUL.
\xhh	Находит указанный в двухзначном шестнадцатеричном виде символ.
\uhhhh	Находит символ юникода по его четырехзначному коду.

Примеры:

var myRegexp = /([A-Za-z]+)=(\d+)/; //Объявление регулярного выражения в стиле #C

var myRegexp = new RegExp("([A-Za-z]+)=(\d+)"); //Объявление регулярного выражения с использованием оператора new

Регулярные выражения произвели прорыв в электронной обработке текста в конце XX века, однако, БАЗИС-скрипты в большей степени ориентированы на выполнение геометрических построений и вычислений. Поэтому подробно регулярные выражения в данном документе на расматриваются. Вместе с тем ниже приводятся несколько примеров обработки текста с использованием регулярных выражений.

Примеры:

var regexp = '1234567890'.replace(/(\d)(?=(\d\d\d)+(?!\d))/g, '$1,'); //Разбиваем числа на троичные (тысячные) группы

history(regexp);

Объект Array и функции работы с массивами

Объект служит для создания массивов и их редактирования. В языке скриптов массив можно создать двумя способами:

- создание массива посредством инициализации.

  var arr = ["value1", 10 , false, "text"]; //Как видно из примера, массив может содержать значения разного типа.

- создание массива при помощи оператора new.

  var arr = new Array("value1","value2","value3");



Массив при программировании скриптов на самом деле представляет собой Std::Map, то есть не обязательно задавать элементы массива с идентификационным номером. Можно задавать их также и с текстовым значением, соответственно и обращаться к таким элементам массива через текстовое значение.

Пример:

var myMassive = new Array(); //Создаю пустой массив

myMassive["first"] = "value1"; //Добавляю значение "value1" в массив myMassive с именем "first"

myMassive["second"] = "value2"; //Добавляю значение "value2" в массив myMassive с именем "second"

var myVar1 = myMassive["first"]; //Обращаюсь к элементу массива с именем "first"

var myVar2 = myMassive["second"]; //Обращаюсь к элементу массива с именем "second"

Для добавления элементов в массив можно использовать следующие методы:

- Добавление элемента(ов) в конец массива: push(value1,value2...valueN);
- Добавление элемента(ов) в начало массива: unshift(value1,value2...valueN);
- Добавление элемента(ов) в указанное место массива (startInsert) с возможной заменой последующих элементов (nmbToRemove): splice(startInsert,nmbToRemove,value1,value2...valueN);
- Также можно дополнять элементы массива следующим образом: arr[3] = "value"; то есть в указанное место массива

Примеры:

var myMassive = new Array("value1", 10 , false, "text"); //Создаю массив с четырьмя значениями

myMassive[4] = "new value"; //Добавляю пятое значение в массив. Обратите внимание на то как происходит номерация массива: размер массива стал равен 5 элементам, а обращение к ним происходит от 0 до 4 соответственно. Результат: myMassive("value1", 10 , false, "text","new value")

myMassive.push("push value1","push value2"); //добавляю в конец массива еще 2 элемента методом push(). Результат: myMassive("value1", 10 , false, "text","new value","push value1","push value2")

myMassive.unshift("some first1"); //Добавляю 1 элемент в начало массива. Результат: myMassive("some first1","value1", 10 , false, "text","new value","push value1","push value2")

myMassive.splice(2,6,"Inserted Value1","Inserted Value2"); //Удаляю элементы массива со 2 по 6 и вставляю два новых элемента. Результат: myMassive("some first1","value1","Inserted Value1","Inserted Value2");

var S = "";

for (var i = 0; i < myMassive.length; ++i)

S += "\"" + myMassive[i] + "\" "; //На экран будет выведен текст: "some first1" "value1" "Inserted Value1" "Inserted Value2"

infomsg(S);

Для изменения порядка массива используется функция reverse();

Пример:

//Изменение порядка массива:

var myMassive = new Array("first","second","third"); //Создаю массив с тремя значениями

var reversedMassive = myMassive.reverse(); //Изменяю порядок массива на обратный

var S = "";

for (var i = 0; i < reversedMassive.length; ++i)

S += "\"" + reversedMassive[i] + "\" "; //На экран будет выведен текст: "third" "second" "first"

infomsg(S);

В предыдущих примерах мы преобразовывали массив в строку с использованием цикла for, однако есть более простой способ получения строки из массива - функция join("connectionSymbol"), где connectionSymbol - любой текст или пробел.

Пример:

//Извлечение строки из массива:

var myMassive = new Array("first","second","third"); //Создаю массив с тремя значениями

var myStr = myMassive.join(" "); //Объединяю элементы массива через пробел в строку

infomsg(myStr);

Для объединения массивов существует функция concat(имя_массива)

Пример:

//Объединение двух массивов

var myMassive1 = new Array("first","second","third"); //Создаю массив с тремя значениями

var myMassive2 = new Array("four","five"); //Создаю массив с двумя значениями

var resultMassive = myMassive1.concat(myMassive2);//Объединяю массивы

var myStr = resultMassive.join(" "); //Объединяю элементы массива через пробел в строку

infomsg(myStr);//Получим следующий вывод: first second third four five

Для сортировки массива используем функцию sort()

Пример:

// Сортировка массива

var myMassive = new Array("B","A","C",0,19,-10); //Создаю массив

var sortedMassive = myMassive.sort(); //Получаю отсортированный массив

var myStr = sortedMassive.join(" "); //Объединяю элементы массива через пробел в строку

infomsg(myStr);//Получим следующий вывод: -10 0 19 A B C

Объект Date

Объект служит для хранения даты и времени. Для получения текущей даты необходимо использовать конструктор: var myDate = new Date(); для обработки других значений даты можно также задать год, месяц, число, часы, минуты, секунды и миллисекунды: var prevDate = new Date(2010,6,18,13,00,30).

Для считывания значений с объекта Date используются функции:

- getYear(),
- getMounth(),
- getDay(),
- getHours(),
- getMinutes(),
- getSeconds(),
- getMilliseconds();

Для установки значений объекта Date используются следующие функции:

- setYear(),
- setMounth(),
- setDay(),
- setHours(),
- setMinutes(),
- setSeconds(),
- setMilliseconds();

Пример:

// Вывод текущего дня недели

var myMassive = new Array("Понедельник","Вторник","Среда","Четверг","Пятница","Суббота","Воскресенье"); //Создаю массив со всеми днями недели

var curDate = new Date(); //Считываю текущую дату

infomsg("Сегодня: " + myMassive[curDate.getDay() - 1]);//Вывожу на экран текущий день недели

Объект Math

Объект позволяет выполнять в скриптах некоторые математические операции. Также объект содержит в себе ряд констант (табл. 3.2).

Табл. 3.2. Математические константы

Константа	Описание
E	Число E. Значение константы Эйлера (2.718281828459045)
LN2	Натуральный логарифм числа 2 (0.6931471805599453)
LN10	Натуральный логарифм числа 10 (2.302585092994046)
LOG2E	Логарифм числа E по основанию 2 (1.4426950408889633)
LOG10E	Логарифм числа E по основанию 10 (0.4342944819032518)
PI	Число PI (3.141592653589793)
SQRT1_2	Квадратный корень числа 1/2 (0.7071067811865476)
SQRT2	Квадратный корень числа 2 (1.4142135623730951

Доступ к этим константам: Math.E; Math.LN2 и т.д.

Модуль числа

Для любого числа возвращает его положительное значение с помощью функции Math.abs(value);

Пример:

var someValue = -133.1527;

var result = Math.abs(someValue); //Извлекаю модуль числа

infomsg("Извлекли модуль числа " + someValue + ". Результат: " + result);//Вывод на экран

Округление чисел

Используется три режима округления чисел:

- Math.ceil(число); – возвращает целое число, большое или равное переданному значению
- Math.floor(число); – возвращает целое число, меньшее или равное переданному значению
- Math.round(число); – округляет число до ближайшего целого значения

Пример:

var someValue = 5.6;

var result1 = Math.ceil(someValue); //Результ: 6

var result2 = Math.floor(someValue); //Результ: 5

var result3 = Math.round(someValue); //Результ: 6

history(result1 + " " + result2 + " " + result3);//Вывод на экран

Определение максимума и минимума

- Math.max(число1,число2); - возвращает большее число
- Math.min(число1,число2); - возвращает меньшее число

Пример:

var someValue1 = 5.6;

var someValue2 = 10.653;

var result1 = Math.max(someValue1,someValue2); //Результ: 10.653

var result2 = Math.min(someValue1,someValue2); //Результ: 5.6

history(result1 + " " + result2);//Вывод на экран

Возведение в степень

Math.pow(число,показатель степени);

Пример:

var result = Math.pow(2,3); //Возвожу 2 в 3-ю степень. Результ: 8

history(result);//Вывод на экран

Вычисление квадратного корня

Math.sqrt(число);

Пример:

var result = Math.sqrt(16); //Извлекли квадратный корень из 16. Результ: 4

history(result);//Вывод на экран

Генератор случайных чисел

Math.random();//Возвращает случайное число от 0 до 1

Пример:

*var result = Math.floor(Math.random()*1000); //Возвращает целое число от 0 до 1000*

history(result);//Вывод на экран

Методы тригонометрии

- Math.cos(rad);//Извлечение косинуса угла, заданного в радианах
- Math.sin(rad);//Извлечение синуса угла, заданного в радианах
- Math.tan(rad);//Извлечение тангенса угла, заданного в радианах
- Math.acos(value);//Извлечение угла в радианах по его косинусу
- Math.asin(value);//Извлечение угла в радианах по его синусу
- Math.atan(value);//Извлечение угла в радианах по его тангенсу

Скрипты не работают с углами, заданными в градусах, а потому прежде чем проводить тригонометрические вычисления, необходимо эти углы перевести в радианы или обратно для результатов обратных тригонометрических функций:

- var rad = angle * Math.PI / 180;//Перевод угла angle в радианы rad.
- var angle = rad / Math.PI * 180;//Перевод угла заданного в радианах rad в градусы angle.

Пример:

//Нахождение синуса угла 270 гр (-90)

*var rad = -90 * Math.PI / 180; //Перевожу угол в радианы*

history(Math.sin(rad)); //Вывод на экран: -1



Объект Math не имеет функции для вычисления котангенса. Ее можно реализовать самому: var ctg = Math.cos(angle) / Math.sin(angle).

Вычисление натурального логарифма

Math.log(value);//Функция возвращает натуральный логарифм числа value

Пример:

var x = Math.log(Math.E); //Натуральный логарифм числа E

history(x); //Вывод на экран: 1



Объект Math не имеет функции для вычисления десятичных логарифмов. Ее можно реализовать самому: var log10 = Math.log(value) / Math.log(10)

Общее для всех модулей

Системные объекты

NewVector

Получить трехмерную точку.

Синтаксис:

Vector (x, y, z);

Входные параметры:

Значения координат x, y, z.

Возвращаемое значение:

Vector

Пример:

p = Vector (100, 100, 200); //Задать точку с координатами x=100, y=100, z=200

NewPoint

Двухмерная точка.

Синтаксис:

Point(x,y);

Входные параметры:

Значения координат x, y.

Возвращаемое значение:

Point.

Пример:

p = Point (100, 100); //Задать точку с координатами x=100, y=100

NewContour

Создать плоский контур.

Синтаксис:

NewContour;

Входные параметры:

Нет.

Возвращаемое значение:

Contour2D

Пример:

Hole = NewContour();

Примечание.

Контур, создаваемый командой NewContour, можно использовать для редактирования контуров с использованием команд Subtraction и Addition.

NewCOMObject

Создать новый COM объект по его типу.

Синтаксис:

NewCOMObject(CLSID);

Входные параметры:

CLSID

Выходные параметры:

Object

Пример:

```
Word = NewCOMObject('Word.Application');
Word.Visible = true;
```

Word.Documents.Add();

Word.Selection.TypeText('Hello Bazis\n');

NewForm

Создать новую экранную форму.

Синтаксис:

NewForm;

Входные параметры:

Нет.

Возвращаемое значение:

ScriptForm

Пример:

DlgBox = NewForm();

Примечание.

Экранная форма может быть создана в визуальном режиме.

Глобальные константы

AxisX

Ось X.

Тип данных:

Vector

AxisY

Ось Y.

Тип данных:

Vector

AxisZ

Ось Z.

Тип данных:

Vector

Axis_X

Ось X, отрицательное направление.

Тип данных:

Vector

Axis_Y

Ось Y, отрицательное направление.

Тип данных:

Vector

Axis_Z

Ось Z, отрицательное направление.

Тип данных:

Vector

Глобальные перечисляемые типы

TProjection3dEnum

Направления обзора

Направления обзора. Совпадают с основными видами.

Значение	Описание
p3dFront	Вид спереди.
p3dLeft	Вид слева.
p3dRight	Вид справа.
p3dTop	Вид сверху.
p3dBottom	Вид снизу.
p3dIsometric	Аксонометрия.

PanelSideEnum

Сторона панели.

Значение	Описание
true	Лицевая сторона.
false	Обратная сторона.

SalonAnimationType

Типы анимации сборок.

Значение	Описание
None	Не учитывается в салоне.
Custom	Учитывается в салоне, не имеет анимации.
DoorLeft	Дверь левая.
DoorRight	Дверь правая.

Значение	Описание
DoorFlap	Дверь откидная.
DoorLift	Дверь подъемная.
SDoorLeft	Дверь купе левая.
SDoorRight	Дверь купе правая.
Drawer	Ящик.
Support	Опора.
Handle	Ручка.
Facade	Фасад.

TextureDir

Ориентация текстуры пластика панели.

Значение	Описание
None	Отсутствует.
Horizontal	Горизонтальная.
Vertical	Вертикальная.

WindowPosition

Позиция формы.

Значение	Описание
Default	Стандартная позиция формы.
Left	Форма пристыковывается слева
Right	Форма пристыковывается справа

FurnSchemeType

Тип схемы расстановки крепежа.

Значение	Описание
WithBase	Тип схемы с фиксированным отступом.
Symmetric	Симметричный тип схемы.
VariableStep	Тип схемы с переменным шагом

SchemeFurnPos

Способы установки фурнитуры.

Значение	Описание
Inside	Установка фурнитуры внутри стыка
Outside	Установка фурнитуры снаружи стыка
Up	Установка фурнитуры вверху стыка (только для стыков с горизонтальной панелью)
Down	Установка фурнитуры внизу стыка (только для стыков с горизонтальной панелью)

ErrorType

Тип ошибки анализа модели

Значение	Описание
ObjIntersection	Пересечение объектов
FastIntersection	Пересечение фурнитуры
FastIncorrect	Неправильная установка фурнитуры
MatNotExists	Материала нет в наличии
MatOutOfStock	Материал отсутствует на складе
PanelTooLarge	Панель невозможно разместить на плите
PlasticTooLarge	Пластик невозможно разместить на панели
PanelNotFixed	Панель не закреплена

DatumMode

Тип монтирования фурнитуры/фрагмента.

Значение	Описание
None	Отсутствует
Face	На плоскость
FaceFace	По двум непараллельным плоскостям
FaceButt	По плоскости и середине торца панели
FaceEdge	По плоскости и ребру
ParallelFaces	На 2 параллельные плоскости
Box	Секция
Scheme	Крепеж по схеме

FurniturePosition

Позиция фурнитуры при установке крепежа по схеме.

Значение	Описание
Inside	Установка фурнитуры внутри стыка
Outside	Установка фурнитуры снаружи стыка
Up	Установка фурнитуры вверху стыка (только для стыков с горизонтальной панелью)
Down	Установка фурнитуры внизу стыка (только для стыков с горизонтальной панелью)

ElementType

Тип 2D элемента.

Значение	Описание
Unknown	Неизвестный
Line	Линия
Arc	Дуга
Circle	Окружность
List	Список элементов
Ellipse	Эллипс

FurnPositionMode

Режим расстановки позиций.

Значение	Описание
All	Раставить позиции заново
New	Раставить позиции у новых объектов
Check	Проверить позиции

Глобальные свойства и методы

Глобальные свойства

system

Системные функции.

Синтаксис:

system;

Тип данных:

System

Model

Структура модели

Синтаксис:

Model;

Тип данных:

Model3D;

Action

Активный скрипт

Синтаксис:

Action;

Тип данных:

Action3D

ActiveMaterial

Текущий материал

Синтаксис:

ActiveMaterial;

Тип данных:

FurnMaterial

AnimationType

Типы анимации сборок и блоков.

Синтаксис:

AnimationType;

Тип данных:

SalonAnimationType

Глобальные методы

prompt

Ввод текстового параметра в диалоге.

Синтаксис:

prompt('<hint>');

Входные параметры:

hint – подсказка, отображаемая в строке состояния.

Тип данных:

String

Выходные параметры:

Текст, введенный в диалоге.

Пример:

```
name  = prompt('Введите имя панели');
alert(name);
```

alert

Вывести на экран сообщение или значения параметров.

Синтаксис:

alert('<текст сообщения>');

Входные параметры:

Текст сообщения, string, или имя параметра, значение которого необходимо вывести на экран. Допускается объединение нескольких параметров при помощи символа +.

Выходные параметры:

Нет.

Пример:

```
alert ('Длина = '+len1);
```

confirm

Вывести на экран диалог подтверждения или отмены.

Синтаксис:

confirm('<текст запроса>');

Входные параметры:

Текст запроса, string.

Тип данных:

Boolean

Пример:

```
confirm('Построить модель?');
```

OpenFurniture

Открыть файл фурнитурного изделия для установки в модель.

Синтаксис:

OpenFurniture(<имя файла>);

Входные параметры:

<имя файла> – имя файла фурнитурного изделия.

Возвращаемый результат:

InFurniture.

Пример:

f = OpenFurniture('C:\\Program Files\\BazisSoft\\Bazis 8\\Samples\\Samples\\Уголок.f3d');



- <имя файла> может включать в себя относительный путь от папки, в которой сохранен скрипт или абсолютный путь. В качестве разделителей элементов пути вместо одиночного символа \ необходимо использовать двойной (\\).
- После добавления фурнитурного изделия необходимо установить его, используя команды Mount или Mount1.

SelectAll

Выделить все объекты.

Синтаксис:

SelectAll();

UnSelectAll

Отменить выделение всех объектов.

Синтаксис:

UnSelectAll();

ViewAll

Показать все.

Синтаксис:

ViewAll();

Описание

Команда позволяет назначить такой максимально возможный масштаб отображения модели, чтобы в окне поместились все объекты.

SetCamera

Установить текущий вид.

Синтаксис:

SetCamera(TProjection3d);

Входные параметры:

TProjection3dEnum

GetPoint

Запрос точки.

Синтаксис:

GetPoint('<hint>'),

Входные параметры:

<hint> – строка запроса, отображаемая в строке состояния.

Возвращаемый результат:

Vector

Пример:

point = GetPoint('Укажите центр отверстия');

GetObject

Запрос объекта модели.

Синтаксис:

GetObject('<hint>'),

Входные параметры:

<hint> – строка запроса, отображаемая в строке состояния.

Возвращаемый результат:

Object3

Пример:

Obj = GetObject('Выберите объект');

GetPanel

Запрос панели.

Синтаксис:

GetPanel('<hint>'),

Входные параметры:

<hint> – строка запроса, отображаемая в строке состояния.

Возвращаемый результат:

Panel

Пример:

Pan1 = GetPanel('Выберите панель');

GetEdge

Запрос выбора ребра, параллельного указанному вектору.

Синтаксис:

GetEdge('<hint>'),

Входные параметры:

<hint> – текст запроса, отображаемый в строке состояния.

Возвращаемый результат:

Edge3

Пример:

Obj = GetEdge('Укажите ребро');

AddPanel

Добавить панель.

Синтаксис:

AddPanel (width, height)

Входные параметры:

width – ширина панели,

height – высота панели.

Возвращаемый результат:

Panel

Пример:

VPanel = AddPanel (400, 600);

Описание

После выполнения команды в модель добавляется панель с заданными параметрами. Панель располагается в плоскости, которая находится ближе других к плоскости, параллельной экрану. Вершина панели находится в начале координат.

AddFrontPanel

Добавить фронтальную панель.

Синтаксис:

AddFrontPanel (x1, y1, x2, y2, z);

Входные параметры:

x1, y1 – координаты начальной точки,

x2, y2 – координаты конечной точки,

z – расстояние от начала координат до панели по оси Z.

Возвращаемый результат:

Panel

Пример:

FPanel = AddFrontPanel (40, 40, 640, 700, 300);

Описание

После выполнения команды в модель добавляется панель с заданными параметрами.

AddHorizPanel

Добавить горизонтальную панель.

Синтаксис:

AddHorizPanel(xl, zl, x2, z2, y);

Входные параметры:

x1, z1 – координаты начальной точки,

x2, z2 – координаты конечной точки,

y – расстояние от начала координат до панели по оси Y.

Возвращаемый результат:

Panel

Пример:

HPanel = AddHorizPanel (40, 40, 640, 700, 300);

Описание

После выполнения команды в модель добавляется панель с заданными параметрами.

AddVertPanel

Добавить вертикальную панель.

Синтаксис:

AddVertPanel(z1, y2, z2, y2, x);

Входные параметры:

z1, y1 – координаты начальной точки,

z2, y2 – координаты конечной точки,

x – расстояние от начала координат до панели по оси X.

Возвращаемый результат:

Panel

Пример:

VPanel = AddVertPanel (40, 40, 640, 700, 300);

Описание

После выполнения команды в модель добавляется панель с заданными параметрами.

AddExtrusion

Добавить тело выдавливания.

Синтаксис:

AddExtrusion(‘name’);

Входные параметры:

name – имя объекта, string.

Возвращаемый результат:

Extrusion.

Пример:

Profile = AddExtrusion ("Профиль");

Описание

После выполнения команды в модель добавляется тело выдавливания с указанным именем. Тело первоначально «пустое». Необходимо задать его контур и толщину.

AddTrajectory

Добавить кинематический элемент (тело по траектории).

Синтаксис:

AddTrajectory(‘name’);

Входные параметры:

name – имя траектории.

Возвращаемый результат:

Trajectory.

Описание

Траектория представляет собой тело, получаемое в результате перемещения контура профиля вдоль контура траектории. После выполнения команды в модель добавляется пустая траектория. Необходимо задать форму контура и траекторию перемещения контура.

Пример:

```
Traj = AddTrajectory('Траектория');
p1 = Vector(0, 0, 0);
p2 = Vector(40, 50, 70);
p3 = Vector(100, 150, 120);
```

Traj.Trajectory2D.AddArc3(p1, p2, p3);

Traj.Contour2D.AddRoundRect(30, 50, 50, 70, 8);

AddBlock

Добавить блок.

Синтаксис:

AddBlock("Имя блока");

Входные параметры:

Имя блока, string.

Возвращаемый результат:

Block

Описание

После выполнения команды в модель добавляется пустой блок с указанным именем.

Пример:

Block = AddBlock ("Блок");

AddAssembly

Добавить сборку.

Синтаксис:

AddAssembly("<Имя сборки>");

Входные параметры:

Имя сборки, string.

Возвращаемый результат:

Assembly

Пример:

Asm = AddAssembly ("Сборка");

Описание:

После выполнения команды в модель добавляется пустая сборка с указанным именем.

AddDraftBlock

Создать полуфабрикат.

Синтаксис:

AddDraftBlock("<name>");

Входные параметры:

name, string.

Возвращаемый результат:

Block

Пример:

```
Draft = AddDraftBlock("Полуфабрикат");
```

Описание:

После выполнения команды в модель добавляется пустая сборка с указанным именем.

AddCopy

Создать копию объекта.

Синтаксис:

```
AddCopy(<obj>);
```

Входные параметры:

obj – копируемый объект.

Выходные параметры:

Object3

DeleteNewObjects

Удалить объекты, ранее созданные в скрипте.

Синтаксис:

```
DeleteNewObjects();
```

DeleteObject

Удалить объект из модели.

Синтаксис:

```
DeleteObject(obj);
```

Входные параметры:

obj – удаляемый объект.

Пример:

```
p1 = AddPanel(100, 200);
p2 = AddPanel(140, 300);
if (confirm("Удалить?")) {
  DeleteObject(p1);
}
```

StartEditing

Начать редактирование объекта, находящегося в модели. Необходимо вызывать команду, чтобы изменения, внесенные в модель в процессе редактирования, могли быть отменены командой **Отменить(Ctrl+z)**.

Синтаксис:

StartEditing(obj);

Входные параметры:

obj – объект модели.

BeginBlock

Начать создание блока. Все объекты, создаваемые после этой команды и до EndBlock попадают внутрь блока.

Синтаксис

BeginBlock("Имя_блока");

Входные параметры:

Имя блока.

Выходные параметры:

Block.

EndBlock

Завершить создание блока. Все объекты, создаваемые после команды BeginBlock и до EndBlock попадают внутрь блока.

Синтаксис

EndBlock();

Пример:

```
BeginBlock("Блок");
AddFrontPanel(0, 0, 50, 50, 0);
AddFrontPanel(0, 0, 50, 50, 100);
EndBlock();
```

NewButtonInput

Создание кнопки на Панели параметров.

Синтаксис:

NewButtonInput (’<hint>’);

Входные параметры:

<hint> – строка запроса, отображаемая в строке состояния.

Выходные параметры:

InButton

Пример:

```
p = NewButtonInput(‘Кнопка1’);
n = NewButtonInput(‘Кнопка2’);
true;
function $input(id)
{
if (p.id == id)
alert(p.id)
else
alert (n.id);
}
```

NewFloatInput

Ввод действительного числа.

Синтаксис:

NewFloatInput (’<hint>’);

Входные параметры:

<hint> – строка запроса, отображаемая в строке состояния.

Выходные параметры:

InFloat.



Допускается ввод математических выражений.

Пример:

```
p = NewFloatInput (‘Введите число’);
```

NewNumberInput

Ввод целого числа.

Синтаксис:

NewNumberInput (’<hint>’);

Входные параметры:

<hint> – строка запроса, отображаемая в строке состояния.

Выходные параметры:

InNumber



Допускается ввод математических выражений, например, 45*3. Результат округляется до целого числа.

Пример:

p = NewNumberInput (‘Введите число’);

NewMaterialInput

Ввод материала.

Синтаксис:

NewMaterialInput (’<hint>’);

Входные параметры:

<hint> – строка запроса, отображаемая в строке состояния.

Выходные параметры:

InMaterial.

Пример:

```
k = NewMaterialInput("Выберите материал");
true;
function $input()
{
alert(k.Thickness);
alert(k.Name);
alert ('Ширина = '+k.Width);
return true;
}
```

NewButtMaterialInput

Ввести кромочный материал.

Синтаксис:

NewButtMaterialInput (’<hint>’);

Входные параметры:

<hint> – строка запроса, отображаемая в строке состояния.

Выходные параметры:

InButtMaterial.

Пример:

```
k = NewButtMaterialInput ("Выберите материал");
```

NewFurnitureInput

Создать элемент управления для выбора фурнитуры.

Синтаксис:

NewFurnitureInput ('<hint>');

Входные параметры:

<hint> – строка запроса, отображаемая в строке состояния.

Выходные параметры:

InFurniture

Пример:

k = NewFurnitureInput ("Выберите фурнитуру");

FormatMatName

Создать схему установки крепежа

Синтаксис:

FormatMatName(<matName>);

Входные параметры:

<matName> – имя материала, string.

Выходные параметры:

string

OrientCamera

Создать схему установки крепежа

Синтаксис:

OrientCamera(dir);

Входные параметры:

<dir> – новое направление взгляда, Vector.

Выходные параметры:

нет

ExtractMatName

Извлечь имя материала из полного имени.

Синтаксис:

ExtractMatName(<matName>);

Входные параметры:

<matName> – полное имя материала, string.

Выходные параметры:

string

ExtractMatCode

Извлечь артикул материала из полного имени.

Синтаксис:

ExtractMatCode(<matName>);

Входные параметры:

<matName> – полное имя материала, string.

Выходные параметры:

string

Geometry2D

Системный объект для реализации вспомогательных геометрических алгоритмов.

Методы:

Intersect – Вычислить точки пересечения двухмерных элементов.

Compare – Сравнить элементы.

Distance – Найти кратчайшее расстояние между элементами.

Area – Найти площадь контура.

Методы

Intersect

Вычислить точки пересечения двухмерных элементов.

Синтаксис:

Intersect(elem1,elem2);

Входные параметры:

elem1,elem2 – пересекающиеся двухмерные элементы, .

Возвращаемое значение:

Array.<Point>

Compare

Сравнить элементы.

Синтаксис:

Compare(elem1,elem2);

Входные параметры:

elem1,elem2 – двухмерные элементы.

Возвращаемое значение:

Boolean

Distance

Найти кратчайшее расстояние между элементами.

Синтаксис:

Distance(elem1,elem2);

Входные параметры:

elem1,elem2 – двухмерные элементы.

Возвращаемое значение:

Number

Area

Найти площадь контура.

Синтаксис:

Area(contour);

Входные параметры:

contour – двухмерный контур, Contour2D.

Возвращаемое значение:

Number

System

Системный объект

Методы:

include – Подключить указанный файл JavaScript.

log – Вывести диагностическое сообщение (для отладки).

fileExists –Проверить, существует ли указанный файл.

writeTextFile – Записать текст в файл.

askWriteTextFile –Записать текст в файл с запросом имени файла.

readTextFile – Считать текст из файла.

askReadTextFile – Считать текст из файла с запросом выбора файла.

secureExec –Выполнить зашифрованный код.

exec –Выполнить внешнюю программу.

askFolder – Открыть диалог выбора папки.

getFileName – Получить имя файла без пути.

askFileNameSave – Открыть диалог сохранения файла.

getFileNameWithoutExtension – Получить имя файла без пути и расширения.

Свойства:

apiVersion – Текущая версия Bazis API.

developerApiVersion – Максимальная версия Bazis API, гарантирующая работу скрипта.

Методы

include

Подключить указанный файл JavaScript.

Синтаксис:

include(<file.js>);

Входные параметры:

<file.js> – Имя подключаемого файла.

log

Вывести диагностическое сообщение (для отладки).

Синтаксис:

log(<msg>);

Входные параметры:

<msg> – Текст сообщения.

fileExists

Проверить, существует ли указанный файл.

Синтаксис:

fileExists(<file>);

Входные параметры:

<file> – имя файла, включая полный путь или путь относительно текущей папки, в которой находится файл скрипта.

writeTextFile

Записать текст в файл.

Синтаксис:

writeTextFile(<file>,<content>);

Входные параметры:

<file> – имя файла, включая полный путь или путь относительно текущей папки, в которой находится файл скрипта.

<content> – содержание файла.

askWriteTextFile

Записать текст в файл с запросом имени файла в диалоге.

Синтаксис:

askWriteTextFile(<ext>,<content>);

Входные параметры:

<ext> – расширение имени файла.

<content> – содержание файла.

readTextFile

Считать текст из файла.

Синтаксис:

readTextFile(<file>);

Входные параметры:

<file> – имя файла, включая полный путь или путь относительно текущей папки, в которой находится файл скрипта.

askReadTextFile

Считать текст из файла с запросом выбора файла в диалоге.

Синтаксис:

askReadTextFile(<ext>);

Входные параметры:

<ext> – расширение имени файла.

secureExec

Выполнить зашифрованный код.

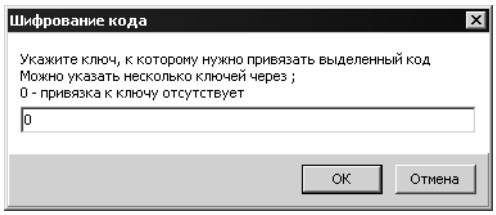
Синтаксис:

secureExec(str);

Входные параметры:

str – зашифрованный текст скрипта.

Метод позволяет выполнить зашифрованный скрипт. Шифровать текст текущего скрипта позволяет команда Редактора скриптов **Правка** – **Шифрование кода**. Перед выполнением команды следует выделить текст скрипта, который необходимо зашифровать. Если не выделено ничего, шифруется скрипт целиком. После вызова команды на экране появится диалог **Шифрование кода**. В этом диалоге можно ввести номер ключа аппаратной защиты системы БАЗИС. Скрипт, «привязанный» таким образом к ключу, может быть выполнен только в модуле БАЗИС-Мебельщик, защищенном этим ключом. Если номер не задан (номер ключа = 0), привязка отсутствует, скрипт может быть выполнен в любом модуле БАЗИС-Мебельщик. Можно ввести несколько номеров ключей, разделяя их знаком <;>.



В результате выполнения команды скрипт автоматически изменяется следующим образом:

- добавляется команда system.secureExec();
- в качестве ее параметра используется зашифрованный текст.

Пример зашифрованного скрипта:

```
system.secureExec("" +
   "AQAA/wF4nG2OMQ7CMAxFfxkYkLiDBwa6lKEj6oAQ" +
   "M0gcgKEtlgMJKi0L6ik4MPy4JVSALcfJz0v8R+hi" +
   "zNq7psrLJyPyUvS+enBZlUcNAweLBGvt/mzRoMQc" +
   "MZaYYqK1YB6p28EbwUzpC/Va+buyorQhL1QNCvaM" +
   "KWQdDqyKfBz+lS8nG64FtkqdqKUDL55vg6t/js7c" +
   "3Xr/EhwJXSbY8e5K9sNnP9M7Ju3ntXgBBekyhA==" +
   "");
```

Все дальнейшие действия со скриптом (сохранение, запуск и т.п.) являются стандартными.

exec

Выполнить внешнюю программу.

Синтаксис:

exec(str);

Входные параметры:

str – полное имя исполняемого файла.

Пример:

system.exec('c:/Windows/system32/calc.exe');

getFileName

Получить имя файла без пути.

Синтаксис:

getFileName(filename);

Входные параметры:

filename – полное имя файла, string.

Возвращаемое значение:

имя файла без пути, string.

getFileNameWithoutExtension

Получить имя файла без пути и расширения.

Синтаксис:

getFileNameWithoutExtension(filename);

Входные параметры:

filename – полное имя файла, string.

Возвращаемое значение:

имя файла без пути и расширения, string.

askFileNameSave

Открыть диалог сохранения файла.

Синтаксис:

askFileNameSave(ext);

Входные параметры:

ext – расширение имени файла, string.

Возвращаемое значение:

полное имя файла, string.

askFolder

Открыть диалог выбора папки.

Синтаксис:

askFolder(caption, defaultFolder);

Входные параметры:

caption – заголовок папки, string

defaultFolder – папка по умолчанию, string

Возвращаемое значение:

имя папки, string.

Свойства

apiVersion

Текущая версия Bazis API

Синтаксис:

apiVersion;

Тип данных:

Number

Примечание.

system.apiVersion = 90 -> Bazis9

system.apiVersion = 85 -> Bazis8

Пример:

```
alert(system.apiVersion);
```

developerApiVersion

Максимальная версия Bazis API, гарантирующая работу скрипта

Синтаксис:

developerApiVersion;

Тип данных:

Number



Версия Bazis API (system.apiVersion) теперь имеет тип Number и подчиняется спецификации «Семантического версионирования» в формате XXYY, где XX - мажор версия и YY - минор версия. Начинается с номера 1001.
Минор версия (YY, в данном случае 01) – увеличивается, когда в API сделаны изменения, совместимые с предыдущими версиями или добавлены функции. Например: скрипт, написанный в версии 1001, будет работать в версии 1003. Но не гарантируется, что скрипт, написанный в версии 1003, будет работать в версии 1001.
Мажор версия (в данном случае 10) – увеличивается, когда в API сделаны изменения, несовместимые с предыдущей версией. Например: скрипт, написанный в версии 10XX с большой долей вероятности может неправильно работать в версии 11XX и наоборот.

Model3D

Список объектов модели.

Свойства:

Selected – Выделенный объект модели.

SelectionCount – Количество выделенных элементов.

Selections – Список выделенных элементов.

Count – Количество объектов в модели.

Objects – Список объектов модели.

Свойства

Selected

Выделенный объект модели. Если выделено несколько объектов, то первый из них.

Синтаксис:

Selected;

Тип данных:

Object3

SelectionCount

Количество выделенных элементов.

Синтаксис:

SelectionCount;

Тип данных:

Number

Selections

Список выделенных элементов.

Синтаксис:

Selections;

Тип данных:

Array.Object3

Count

Количество объектов в модели.

Синтаксис:

Count;

Тип данных:

Number

Objects

Список объектов модели

Синтаксис:

Objects;

Тип данных:

Array.Object3

Action3D

Текущий скрипт.

Свойства

Interactive – Возможность взаимодействия с пользователем.

MouseX – Координата X позиции курсора мыши на экране.

MouseY – Координата Y позиции курсора мыши на экране.

Pos3 – Текущая позиция курсора в окне модели

ViewDir – Нормаль к текущему виду.

UpDir – Вектор вверх текущего вида.

RightDir – Вектор вправо текущего вида.

ShowPoints – Разрешать подсвечивать точки.

ShowEdges – Разрешать подсвечивать ребра.

Hint – Установить строку подсказки.

ErrorHint –Установить сообщение об ошибке.

BlinkHint –Установить мигающую подсказку.

OnClick – Обработчик щелчка мыши.

OnMove – Обработчик перемещения курсора мыши.

OnFinish – Обработчик завершения работы скрипта.

Properties – Набор редактируемых свойств.

OnRayTraceFinished – Обработчик завершения тонирования изображения.

ModelFilename – Имя файла текущей модели.

Методы

Continue – Продолжить вызывать прерывания по завершению основного тела скрипта, не завершая команды.

Commit – Применить изменения в модели внесенные в скрипте.

Finish – Завершить команду.

Cancel – Отменить команду.

AsyncExec – Выполнить функцию, в которой доступны запросы Get*.

BeginOrtho3 –Включить режим ортогональных построений относительно точки.

EndOrtho3 – Отключить режим ортогональных построений относительно точки.

CursorToClosestPoint –Сдвинуть курсор к ближайшей точке привязки.

CursorToClosestLine –Сдвинуть курсор к ближайшей линии.

CursorToMiddleOfLine –Сдвинуть курсор к ближайшей середине линии.

Find3DPoint – Найти точку на модели в текущем положении курсора мыши.

Find3DPointXZPlane – Найти точку на плоскости X0Z.

Get3DObject – Найти объект под курсором мыши.

RayTraceScene – Запустить тонирование изображения.

LoadModel – Загрузить модель из файла.

SaveModel – Сохранить модель в файл.

NewModel – Создать новую модель.

NewFurniture – Создать новую фурнитуру.

NewFragment – Создать новый фрагмент.

Revert – Отменить изменения в модели, внесенные в скрипте.

ArrangePositions – Расставить позиции.

ReplaceFurniture – Заменить фурнитуру.

ChooseMaterial – Вызов окна выбора материала из базы данных материалов.

Свойства

Interactive

Возможность взаимодействия с пользователем. Если равно false, то запрещены любые функции взаимодействия с пользователем.

Синтаксис:

Interactive;

Тип данных:

Boolean

MouseX

Координата X позиции курсора мыши на экране.

Синтаксис:

MouseX;

Тип данных:

Number

MouseY

Координата Y позиции курсора мыши на экране.

Синтаксис:

MouseY;

Тип данных:

Number

Pos3

Текущая позиция курсора в окне модели.

Синтаксис:

Pos3;

Тип данных:

Vector

ViewDir

Нормаль к текущему виду

Синтаксис:

ViewDir;

Тип данных:

Vector

UpDir

Вектор вверх текущего вида.

Синтаксис:

Updir;

Тип данных:

Vector

RightDir

Вектор вправо текущего вида

Синтаксис:

RightDir;

Тип данных:

Vector

ShowPoints

Разрешать подсвечивать точки.

Синтаксис:

ShowPoints;

Тип данных:

Boolean

ShowEdges

Разрешать подсвечивать ребра.

Синтаксис:

ShowEdges;

Тип данных:

Boolean

Hint

Установить строку подсказки.

Синтаксис:

Hint;

Тип данных:

String

ErrorHint

Установить сообщение об ошибке.

Синтаксис:

ErrorHint;

Тип данных:

String

BlinkHint

Установить мигающую подсказку.

Синтаксис:

BlinkHint;

Тип данных:

String

OnClick

Обработчик щелчка мыши.

Синтаксис:

OnClick;

Тип данных:

Function

Пример:

```
Action.OnClick = function() {
   alert('Щелчок в области модели ' + Action.MouseX + ' ' + Action.MouseY)
};
```

OnMove

Обработчик перемещения курсора мыши.

Синтаксис:

OnMove;

Тип данных:

Function

Пример:

```
Action.OnMove = function() {
alert('Перемещение курсора ')
};
```

OnStart

Обработчик начала работы скрипта. Вызывается после загрузки значений свойств .

Синтаксис:

OnStart;

Тип данных:

Function

OnFinish

Обработчик завершения работы скрипта.

Синтаксис:

OnFinish;

Тип данных:

Function

Properties

Набор редактируемых свойств.

Синтаксис:

Properties;

Тип данных:

RootProperties

OnRayTraceFinished

Обработчик завершения тонирования изображения.

Синтаксис:

OnRayTraceFinished;

Тип данных:

Function

ModelFilename

Имя файла текущей модели.

Синтаксис:

ModelFilename;

Тип данных:

string

Методы

Continue

Продолжить обработку прерываний после завершения основного тела скрипта, не завершая скрипта.

Синтаксис:

Continue();

Пример:

```
x = NewButtonInput("Кнопка");
Action.Continue();
```

Commit

Применить изменения в модели внесенные в скрипте, не завершая скрипта.

Синтаксис:

Commit();

Пример:

```
Commit();
```

Finish

Завершить работу скрипта.

Синтаксис:

Finish();

Cancel

Завершить работу скрипта, отказавшись от изменений.

Синтаксис:

Cancel();

AsyncExec

Выполнить функцию, в которой доступны запросы Get*.

Синтаксис:

AsyncExec(func);

Входные параметры:

func – функция для выполнения (Function).

BeginOrtho3

Включить режим ортогональных построений относительно точки.

Синтаксис:

BeginOrtho3(point);

Входные параметры:

point – точка (Vector)

Пример:

```
p = AddPanel(100, 100);//установить панель
pos = GetPoint("Укажите точку");//указать точку для позиционирования панели
Action.BeginOrtho3(pos);//включить режим ортогональных построений.
Action.Continue();
function $move() {
  p.Position = Action.Pos3;
}
```

Примечание.



- В приведенном примере панель перемещается за курсором мыши. После включения режима ортогональных построений если угол между направлением от точки *pos* на текущее положение курсора и воображаемой линией, параллельной одной из осей координат, будет меньше угла, заданного при настройке зоны ортогональности, курсор и, соответственно, точка привязки панели, расположатся на этой воображаемой линии.
- Чтобы отключить режим ортогональных построений, необходимо использовать метод EndOrtho3.

EndOrtho3

Выключить режим ортогональных построений относительно точки (см. метод BeginOrtho3).

Синтаксис:

EndOrtho3();

CursorToClosestPoint

Сдвинуть курсор к ближайшей точке привязки.

Синтаксис:

CursorToClosestPoint();

CursorToClosestLine

Сдвинуть курсор к ближайшей линии

Синтаксис:

CursorToClosestLine();

CursorToMiddleOfLine

Сдвинуть курсор к ближайшей середине линии.

Синтаксис:

CursorToMiddleOfLine();

Find3DPoint

Найти точку на модели в текущем положении курсора мыши.

Синтаксис:

Find3DPoint();

Возвращаемый результат:

Vector

Find3DPointXZPlane

Найти точку на плоскости X0Z.

Синтаксис:

Find3DPointXZPlane();

Возвращаемый результат:

Vector

Get3Dobject

Найти объект под курсором мыши.

Синтаксис:

Get3DObject();

Возвращаемый результат:

Object3

RayTraceScene

Запустить тонирование изображения.

Синтаксис:

RayTraceScene();

LoadModel

Сохранить модель в файл.

Синтаксис:

LoadModel(filename);

Входные параметры:

filename (string)

Возвращаемый результат:

boolean

SaveModel

Сохранить модель в файл.

Синтаксис:

SaveModel(filename);

Входные параметры:

filename (string)

NewModel

Создать новую модель.

Синтаксис:

NewModel();

NewFurniture

Создать новую фурнитуру.

Синтаксис:

NewFurniture();

NewFragment

Создать новый фрагмент.

Синтаксис:

NewFragment();

Revert

Отменить изменения в модели, внесенные в скрипте.

Синтаксис:

Revert();

ArrangePositions

Расставить позиции.

Синтаксис:

ArrangePositions(Mode, Model);

Входные параметры:

Mode(FurnPositionMode) – режим расстановки

Model(List3D) – структурный объект

Возвращаемый результат:

boolean

ReplaceFurniture

Заменить фурнитуру.

Синтаксис:

ReplaceFurniture(Old, New, Fasteners);

Входные параметры:

Old – список названий старой фурнитуры, string

New – список значений новой фурнитуры, InfFurniture

Fasteners – список объектов для замены, Object3

Возвращаемый результат:

нет

ChooseMaterial

Вызов окна выбора материала из базы данных материалов.

Синтаксис:

ChooseMaterial();

Возвращаемый результат:

Полное имя материала, string.

ScriptProperty

Набор редактируемых свойств.

Свойства:

Name – Имя свойства.

Enabled – Возможность изменения свойства пользователем.

ChildrenEnabled – Доступность вложенных свойств для редактирования.

Visible – Видимость свойства в панели свойств.

OnChange – Обработчик изменения свойства и вложенных свойств.

Count – Количество вложенных свойств первого уровня вложенности.

Items – Список вложенных свойств первого уровня вложенности.

Expanded – Развернуты ли вложенные свойства.

OnDeactivate – Обработчик деактивации свойства или меню.

OnValueValidate – Проверка корректности значения.

ValueValid – Флаг корректности введенного значения, выставляется обработчиком OnValueValidate.

NameEditable – Разрешить редактирование имени (для создания таблиц).

BackColor – Цвет фона

PopupMenu – Создать контекстно-зависимое меню.

DropDownMenu – Создать раскрывающийся список объектов.

OnActivate – Обработчик активации свойства пользователем

OnValueChange – Обработчик изменения свойства

Tag – Пользовательское число

Методы:

Clear – Очистить вложенные свойства.

NewCOMObject – Создать новый COM объект по его типу.

Save – Сохранить введенные пользователем значения свойств в файле формата xml.

Load – Загрузить значения свойств, сохраненные методом Save из файла формата xml.

NewGroup – Создать вложенную группу свойств.

NewImage – Создать вложенную группу свойств с рисунком**.**

NewString – Создать свойство типа Строка.

NewBool – Создать свойство типа Да/Нет.

NewNumber – Создать свойство типа Число.

NewButton – Создать свойство с кнопкой редактирования.

NewMaterial – Создать свойство типа Материал.

NewButt – Создать свойство типа Материал кромки.

NewFurniture – Создать свойство типа Фурнитура.

NewColor – Создать свойство типа цвет.

NewSeparator – Создать разделитель.

Validate – Проверить значение свойства и вложенных свойств')

Свойства

Name

Имя свойства.

Синтаксис:

Name;

Тип данных:

String

Enabled

Возможность изменения свойства пользователем.

Синтаксис:

Enabled;

Тип данных:

Boolean

ChildrenEnabled

Доступность вложенных свойств для редактирования.

Синтаксис:

ChildrenEnabled;

Тип данных:

Boolean

Visible

Видимость свойства в панели свойств.

Синтаксис:

Visible;

Тип данных:

Boolean

OnChange

Синтаксис:

Обработчик изменения свойства и вложенных свойств.

Синтаксис:

OnChange;

Count

Количество вложенных свойств первого уровня вложенности.

Синтаксис:

Count;

Тип данных:

Number

Items

Свойство первого уровня вложенности.

Синтаксис:

Items[i];

Входные параметры:

i – индекс свойства, начиная с 0.

Тип данных:

ScriptProperty

Expanded

Развернуты ли вложенные свойства.

Синтаксис:

Expanded;

Тип данных:

Boolean

OnDeactivate

Обработчик деактивации свойства или меню.

Синтаксис:

OnDeactivate;

Тип данных:

Function

OnValueValidate

Проверка корректности значения.

Синтаксис:

OnValueValidate;

Тип данных:

Function

ValueValid

Флаг корректности введенного значения, выставляется обработчиком OnValueValidate.

Синтаксис:

ValueValid;

Тип данных:

Boolean

NameEditable

Разрешить редактирование имени (для создания таблиц).

Синтаксис:

NameEditable;

Тип данных:

Boolean

BackColor

Цвет фона

Синтаксис:

BackColor;

Тип данных:

Number

PopupMenu

Всплывающее меню.

Синтаксис:

PopupMenu;

Тип данных:

ScriptMenu

DropDownMenu

Выпадающее меню.

Синтаксис:

DropDownMenu;

Тип данных:

ScriptMenu

OnActivate

Обработчик активации свойства пользователем.

Синтаксис:

OnActivate;

Тип данных:

Function

OnValueChange

Обработчик изменения свойства.

Синтаксис:

OnValueChange;

Тип данных:

Function

Tag

Пользовательское число.

Синтаксис:

Tag;

Тип данных:

Number

Методы

Clear

Очистить вложенные свойства.

Синтаксис:

Clear();

NewCOMObject

Создать новый COM объект по его типу.

Синтаксис:

NewCOMObject(CLSID);

Входные параметры:

CLSID

Возвращаемое значение:

IDispatch

Save

Сохранить введенные пользователем значения свойств в файле формата xml.

Синтаксис:

Save('<имя файла>.xml');

Входные параметры:

<имя файла>.xml



- Параметр <имя файла> может включать в себя относительный путь от папки, в которой сохранен скрипт или абсолютный путь. В качестве разделителей элементов пути вместо одиночного символа \ необходимо использовать двойной (\\).
- Набор значений, сохраненный в файле, можно загрузить, используя метод Load.

Load

Загрузить значения свойств, сохраненные методом Save из файла формата xml.

Синтаксис:

Load('<имя файла>.xml');

Входные параметры:

<имя файла>.xml



Параметр <имя файла> может включать в себя относительный путь от папки, в которой сохранен скрипт или абсолютный путь. В качестве разделителей элементов пути вместо одиночного символа \ необходимо использовать двойной (\\).

NewGroup

Создать вложенную группу свойств.

Синтаксис:

NewGroup('<GroupName>');

Входные параметры:

<GroupName> – имя создаваемой группы.

Выходные параметры:

ScriptGroupProperty

NewImage

Создать вложенную группу свойств с рисунком.

Синтаксис:

NewImage:('<GroupName>','imagefile');

Входные параметры:

<GroupName> – имя группы.

imagefile – имя файла рисунка.

Выходные параметры:

ScriptGroupProperty



- Параметр imagefile может включать в себя относительный путь от папки, в которой сохранен скрипт или абсолютный путь. В качестве разделителей элементов пути вместо одиночного символа \ необходимо использовать двойной (\\).
- Поддерживаются следующие форматы графических файлов:
  - векторные:
  - Windows MetaFile (расширение .wmf, .wmz);
  - Enhanced MetaFile (расширение .emf, .emz);
  - растровые:
  - JPEG (расширение .jpg);
  - Windows Bitmap (расширение .jpg);
  - Portable Network Graphic (расширение .png);
  - Tagged Image File Format (расширение .tif).

NewString

Создать свойство типа Строка.

Синтаксис:

NewString('propName', 'default_val');

Входные параметры:

propName – Имя свойства.

default_val – Значение свойства по умолчанию.

Выходные параметры:

ScriptStringProperty

NewBool

Создать свойство типа Да/Нет.

Синтаксис:

NewBool('propName');

Входные параметры:

propName – Имя свойства.

Выходные параметры:

ScriptBooleanProperty

NewNumber

Создать свойство типа Число.

Синтаксис:

NewNumber('propName');

Входные параметры:

propName – Имя свойства.

Выходные параметры:

ScriptNumberProperty

NewButton

Создать свойство с кнопкой редактирования.

Синтаксис:

NewButton('propName','default_val'),

Входные параметры:

propName – Имя свойства.

default_val – Значение свойства по умолчанию.

Выходные параметры:

ScriptButtonProperty

Примечание.

Необходимо создать функцию-обработчик нажатия кнопки.

Пример:

```
o = Action.Properties;
m = o.NewGroup('Материал');
g = m.NewMaterial('Верхние полки');
v12 = g.NewButton('Имя фрагмента', 'test.frw');
v12.OnClick = function() {v12.Value = system.askFileName('frw')};
```

NewMaterial

Создать свойство типа Материал.

Синтаксис:

NewMaterial('propName');

Входные параметры:

propName – Имя свойства.

Выходные параметры:

ScriptMaterialProperty

NewButt

Создать свойство типа Материал кромки.

Синтаксис:

NewButt('propName');

Входные параметры:

propName – Имя свойства.

Выходные параметры:

ScriptButtProperty

NewFurniture

Создать свойство типа Фурнитура.

Синтаксис:

NewFurniture('propName');

Входные параметры:

propName – Имя свойства.

Выходные параметры:

ScriptFurnitureProperty

NewColor

Создать свойство типа Цвет.

Синтаксис:

NewColor('propName');

Входные параметры:

propName – Имя свойства.

Выходные параметры:

ScriptColorProperty

NewSeparator

Создать разделитель.

Синтаксис:

NewSeparator();

Выходные параметры:

ScriptProperty

Validate

Проверить значение свойства и вложенных свойств.

Синтаксис:

Validate();

Тип данных:

Boolean

ScriptGroupProperty

Группа свойств скрипта.

Свойства

Image – путь к файлу изображения для кнопки.

MaxHeight – максимальная высота.

Scrollable – lобавление полосы прокрутки.

Свойства

Image

Путь к файлу изображения для кнопки.

Синтаксис:

Image;

Тип данных:

String

MaxHeight

Максимальная высота картинки.

Синтаксис:

MaxHeight;

Тип данных:

Number



Если значение свойства не задано, высота рассчитывается автоматически. Ширина картинки равна ширине колонки **Значение** панели **Свойства**, при изменении ширины колонки высота изменяется таким образом, чтобы сохранились пропорции картинки. Если значение свойства задано, то при автоматическом масштабировании высота картинкине превысит этого значения.

Scrollable

Добавление полосы прокрутки.

Синтаксис:

Scrollable;

Тип данных:

boolean

Принимаемые значения

True – добавить на компонент типа «группа» полосы прокрутки;

False – не добавлять полосы прокрутки;

ScriptStringProperty

Свойство типа String

Свойства:

Value – значение свойства.

Свойства

Value

Строковое значение.

Синтаксис:

Value;

Тип данных:

String

ScriptBooleanProperty

Свойство типа Boolean

Свойства:

Value – значение свойства.

Свойства.

Value

Значение свойства.

Синтаксис:

Value;

Тип данных:

Boolean

ScriptNumberProperty

Свойство типа Number

Свойства:

MaxValue – значение свойства.

MinValue – значение свойства.

Value – значение свойства.

ValueStep – значение свойства.

Свойства

MaxValue

Максимальное значение свойства.

Синтаксис:

MaxValue;

Тип данных:

Number

Примечание. Значение числового поля можно задавать при помощи ползунка. Чтобы он был виден, необходимо задать значение MaxValue больше, чем MinValue.

MinValue

Минимальное значение свойства.

Синтаксис:

MinValue;

Тип данных:

Number

Примечание. Значение числового поля можно задавать при помощи ползунка. Чтобы он был виден, необходимо задать значение MinValue больше, чем MaxValue.

ValueStep

Примечание. Значение числового поля можно задавать при помощи ползунка. Значение ValueStep определяет расстояние между делениями ползунка.

Синтаксис:

ValueStep;

Тип данных:

Number

Value

Значение свойства.

Синтаксис:

Value;

Тип данных:

Number

ScriptButtonProperty

Обработчик нажатия кнопки.

Свойства:

Value – отображаемое имя параметра.

OnClick – функция-обработчик нажатия на кнопку.

Свойства

Value

Отображаемое имя параметра

Синтаксис:

Value;

Тип данных:

String

OnClick

Функция-обработчик нажатия на кнопку.

Синтаксис:

OnClick;

Тип данных:

Function

ScriptMaterialProperty

Свойство типа Материал

Свойства:

Width – Ширина листа.

Методы:

SetActive – Установить материал активным. Все последующие элементы будут построены из этого материала.

Свойства

Width

Ширина листа.

Синтаксис:

Width;

Тип данных:

Number

Методы

SetActive

Установить материал активным. Все последующие элементы будут построены из этого материала.

Синтаксис:

SetActive();

ScriptButtProperty

Свойство типа Материал кромки

Методы:

SetActive – Установить материал кромки активным. Все последующие элементы будут построены из этого материала.

Свойства:

Thickness – Толщина кромки.

Width – Ширина ленты.

Свойства

Thickness

Толщина кромки.

Синтаксис:

Thickness;

Тип данных:

Number

Width

Ширина ленты.

Синтаксис:

Width;

Тип данных:

Number

Методы

SetActive

Установить материал кромки активным. Все последующие элементы будут построены из этого материала.

Синтаксис:

SetActive();

ScriptFurnitureProperty

Свойство типа Фурнитура

Свойства:

Value – фурнитура.

Свойства:

Value

Фурнитура.

Синтаксис:

ScriptFurnitureProperty;

Тип данных:

InFurniture

ScriptColorProperty

Свойство типа Цвет

Свойства:

Value – индекс цвета.

Свойства

Value

Синтаксис:

Value;

Тип данных:

Number

ScriptMenu

Объект типа всплывающее или выпадающее меню

Свойства:

Name – Имя свойства

OnChange – Обработчик изменения свойства и вложенных свойств

Count – Количество вложенных свойств

Items – Список вложенных свойств

Store – Флаг, сохраняется ли свойство в файл

Методы:

Clear – Очистить вложенные свойства

Save – Сохранить введенные пользователем данные в файле xml

Load – Загрузить значения полей из файла xml

NewGroup – Создать вложенную группу свойств

NewString – Создать свойство типа Строка

NewBool – Создать свойство типа Boolean

NewNumber – Создать числовое свойство

NewButton – Создать кнопку

Свойства

Name

Имя свойства

Синтаксис:

Name;

Тип данных:

String

OnChange

Обработчик изменения свойства и вложенных свойств

Синтаксис:

OnChange;

Тип данных:

Function

Count

Количество вложенных свойств

Синтаксис:

Count;

Тип данных:

Number

Items

Список вложенных свойств

Синтаксис:

Items;

Тип данных:

Array.ScriptProperty

Store

Сохраняется ли свойство в файл

Синтаксис:

Store;

Тип данных:

Boolean

Методы

Clear

Очистить вложенные свойства.

Синтаксис:

Clear();

Save

Сохранить введенные пользователем данные в файле xml.

Синтаксис:

Save('<имя файла>.xml');

Входные параметры:

<имя файла>.xml



- Параметр <имя файла> может включать в себя относительный путь от папки, в которой сохранен скрипт или абсолютный путь. В качестве разделителей элементов пути вместо одиночного символа \ необходимо использовать двойной (\\).
- Набор значений, сохраненный в файле, можно загрузить, используя метод Load.

Load

Загрузить значения свойств, сохраненные методом Save из файла формата xml.

Синтаксис:

Load('<имя файла>.xml');

Входные параметры:

<имя файла>.xml



Параметр <имя файла> может включать в себя относительный путь от папки, в которой сохранен скрипт или абсолютный путь. В качестве разделителей элементов пути вместо одиночного символа \ необходимо использовать двойной (\\).

NewGroup

Создать вложенную группу свойств

Синтаксис:

NewGroup();

Входные параметры:

Имя группы

Тип данных:

ScriptGroupProperty

NewString

Создать свойство строкового типа

Синтаксис:

NewString();

Тип данных:

ScriptGroupProperty

NewBool

Создать свойство типа Boolean

Синтаксис:

NewBool();

Тип данных:

ScriptBooleanProperty

NewNumber

Создать свойство типа число

Синтаксис:

NewNumber();

Тип данных:

ScriptNumberProperty

NewButton

Создать кнопку

Синтаксис:

NewButton();

Тип данных:

ScriptButtonProperty

ScriptForm

Пользовательская экранная форма, позволяющая задавать параметры объектов.

Свойства:

Properties – Набор редактируемых свойств
Caption – Заголовок формы
Width – Ширина формы
Height – Высота формы
MinWidth – Минимальная ширина формы
MinHeight – Минимальная высота формы
Visible – Видимость формы
Left – Положение левого края формы
Top – Положение верхнего края формы
OKButton – Показывать кнопку **OK** на форме
OKButtonCaption – Текст кнопки **OK**
CancelButton – Показывать кнопку **Отмена** на форме
CancelButtonCaption – Текст кнопки **Отмена**
OnClose – Обработчик закрытия формы
OnShow – Обработчик открытия формы
Resizable – Возможность изменять размеры формы
Dockable – Возможность пристыковывать не модальную форму
OnOkButtonClick – Обработчик нажатия на кнопку **OK**
OnCancelButtonClick – Обработчик нажатия на кнопку **Cancel**

Методы:

Close – Закрыть форму
Show – Показать форму
ShowModal – Показать модальную форму

Свойства

Properties

Набор редактируемых свойств

Синтаксис:

Properties;

Тип данных:

ScriptProperty

Caption

Заголовок формы

Синтаксис:

Caption;

Тип данных:

String

Width

Ширина формы

Синтаксис:

Width;

Тип данных:

Number

Height

Высота формы

Синтаксис:

Height;

Тип данных:

Number

MinWidth

Минимальная ширина формы

Синтаксис:

MinWidth;

Тип данных:

Number

MinHeight

Минимальная высота формы

Синтаксис:

MinHeight;

Тип данных:

Number

Visible

Видимость формы

Синтаксис:

Visible;

Тип данных:

Boolean

Left

Положение левого края формы

Синтаксис:

Left;

Тип данных:

Number

Top

Положение верхнего края формы

Синтаксис:

Top;

Тип данных:

Number

OKButton

Показывать кнопку **OK** на форме

Синтаксис:

OKButton;

Тип данных:

Boolean

OKButtonCaption

Текст кнопки **OK**

Синтаксис:

OKButtonCaption;

Тип данных:

String

CancelButton

Показывать кнопку **Отмена** на форме

Синтаксис:

CancelButton;

Тип данных:

Boolean

CancelButtonCaption

Текст кнопки **Отмена**

Синтаксис:

CancelButtonCaption;

Тип данных:

String

OnClose

Обработчик закрытия формы

Синтаксис:

OnClose;

Тип данных:

Function

OnShow

Обработчик открытия формы

Синтаксис:

OnShow;

Тип данных:

Function

Resizable

Возможность изменять размеры формы

Синтаксис:

Resizable;

Тип данных:

Boolean

Dockable

Возможность пристыковывать немодальную форму

Синтаксис:

Dockable;

Тип данных:

Boolean

OnOkButtonClick

Обработчик нажатия на кнопку **OK**

Синтаксис:

OnOkButtonClick;

Тип данных:

Function

OnCancelButtonClick

Обработчик нажатия на кнопку **Cancel**

Синтаксис:

OnCancelButtonClick;

Тип данных:

Function

Методы.

Show

Показать форму

Синтаксис:

ScriptForm.Show(WindowPosition);

Входные параметры:

WindowPosition

ShowModal

Показать модальную форму

Синтаксис:

ScriptForm.ShowModal();

Возвращаемое значение :

Boolean

Close

Закрыть форму

Синтаксис:

ScriptForm.Close();

FurnMaterial

Материал.

Свойства

Name – Наименование материала.

Thickness – Толщина материала.

Width – Ширина материала.

Методы

Make – Создать материал по наименованию и толщине или ширине.

Свойства

Name

Наименование материала

Синтаксис

Name();

Тип данных

String

Thickness

Толщина материала.

Синтаксис

Thickness();

Тип данных

Number

Width

Ширина материала.

Синтаксис

Width();

Тип данных

Number

Методы.

Make

Создать материал по наименованию и толщине или ширине.

Синтаксис:

FurnMaterial.Make(name, thickness,[width]);

Входные параметры:

name – наименование материала,

thickness – толщина материала,

width (необязательный параметр) – ширина материала.

Vector

Точка в трехмерном пространстве.

Свойства:

x: – координата X точки,

y: – координата Y точки,

z: – координата Z точки,

Свойства:

x

Координата X точки.

Синтаксис:

x;

Тип данных:

Number

y

Координата Y точки.

Синтаксис:

y;

Тип данных:

Number

z

Координата Z точки.

Синтаксис:

z;

Тип данных:

Number

Point

Точка на плоскости.

Свойства:

x – Координата X точки.

y – Координата Y точки.

Свойства:

x

Координата X точки.

Синтаксис:

x;

Тип данных:

Number

y

Координата Y точки на плоскости.

Синтаксис:

y;

Тип данных:

Number

Edge3

Ребро.

Свойства:

First – Начало ребра в ЛСК.

Last – Конец ребра в ЛСК.

GFirst – Начало ребра.

GLast – Конец ребра.

Свойства

First

Начало ребра в ЛСК.

Тип данных:

Vector

Пример:

Last

Конец ребра в ЛСК.

Тип данных:

Vector

Пример:

GFirst

Начало ребра в глобальной системе координат.

Тип данных:

Vector

Пример:

GLast

Конец ребра в глобальной системе координат.

Тип данных:

Vector

Пример:

Пример:

```
Edge = GetEdge("Укажите границу", AxisX);
alert(Edge.First.x);
alert(Edge.Last.x);
alert(Edge.GFirst.x);
alert(Edge.GLast.x);
```

Object3

Трехмерный объект.

Является родительским для Panel, Extrusion, Trajectory, List3D.

Свойства:

Name – Наименование.

ArtPos – Позиция.

Owner – Родитель объекта.

Visible – Видимость объекта.

Selected – Является ли объект выделенным.

List – Является ли объект структурным.

AsList – Привести объект к структурному.

AsPanel – Привести объект к типу панели.

Position – Положение объекта.

PositionX – Координата x.

PositionY – Координата y.

PositionZ – Координата z.

SetDefaultTransform – Установить нулевые положение и ориентацию объекта.

GSize – Размер модели (расстояние между GMax и GMin).

GMin – Вершина габаритного параллелепипеда, наименее удаленная от начала родительской системы координат.

GMax – Вершина габаритного параллелепипеда, наиболее удаленная от начала родительской системы координат.

GabMin – Вершина габаритного параллелепипеда, наименее удаленная от начала глобальной системы координат.

GabMax – Вершина габаритного параллелепипеда, наиболее удаленная от начала глобальной системы координат.

UserPropCount – Количество пользовательских свойств.

UserProperty – Значения свойства по его имени или индексу.

UserPropertyName – Названия свойства.

Методы:

SetDefaultTransform – Установить нулевые положение и ориентацию объекта.

Translate – Сместить объект.

Rotate – Повернуть вокруг заданной оси.

TranslateGCS – Сместить объект в глобальной СК.

RotateGCS – Повернуть объект в глобальной СК.

RotateX – Повернуть объект вокруг оси X.

RotateY – Повернуть объект вокруг оси Y.

RotateZ – Повернуть объект вокруг оси Z.

Orient – Развернуть объект относительно двух осей.

OrientGCS –Развернуть объект относительно двух осей в глобальной СК.

Reflect – Отразить объект относительно плоскости.

ToObject – Преобразовать координаты точки из глобальной СК в ЛСК объекта.

ToGlobal – Преобразовать координаты точки из СК объекта в глобальную СК.

NToObject – Преобразовать нормаль в ЛСК объекта.

NToGlobal – Преобразовать нормаль из ЛСК объекта.

Build – Перестроить объект после изменения его свойств.

UserPropCount – Количество пользовательских свойств.

UserProperty – Значения свойства по его имени или индексу.

UserPropertyName – Названия свойств.

IsOwner – Определить, является ли объект obj родителем объекта.

Свойства:

Name

Наименование.

Синтаксис:

Name;

Тип данных:

String

ArtPos

Позиция.

Синтаксис:

ArtPos;

Тип данных:

String

Owner

Родитель объекта.

Синтаксис:

Owner;

Тип данных:

List3D

Visible

Видимость объекта

Синтаксис:

Visible;

Тип данных:

Boolean

Selected

Является ли объект выделенным.

Синтаксис:

Selected;

Тип данных:

Boolean

List

Является ли объект структурным.

Синтаксис:

List;

Тип данных:

Boolean

AsList

Привести объект к структурному.

Синтаксис:

AsList;

Тип данных:

List3D

AsPanel

Привести объект к типу панели.

Синтаксис:

AsPanel;

Тип данных:

Panel

Position

Положение объекта.

Синтаксис:

Position;

Тип данных:

Vector

PositionX

Координата X.

Синтаксис:

PositionX;

Тип данных:

Number

PositionY

Координата Y.

Синтаксис:

PositionY;

Тип данных:

Number

PositionZ

Координата Z.

Синтаксис:

PositionZ;

Тип данных:

Number

GMax

Вершина габаритного параллелепипеда, наиболее удаленная от начала системы координат родительского объекта.

Синтаксис:

GMax;

Тип данных:

Vector

Пример:

GMin

Вершина габаритного параллелепипеда, наименее удаленная от начала системы координат родительского объекта.

Синтаксис:

GMin;

Тип данных:

Vector

Пример:

GSize

Длина диагонали габаритного параллелепипеда объекта, расстояние от GMax до Gmin.

Синтаксис:

GSize;

Тип данных:

Vector

Пример:

GabMin

Вершина габаритного параллелепипеда, наименее удаленная от начала глобальной системы координат.

Синтаксис:

GabMin;

Тип данных:

Vector

Пример:

GabMax

Вершина габаритного параллелепипеда, наиболее удаленная от начала глобальной системы координат.

Синтаксис:

GabMax;

Тип данных:

Vector

Пример:

UserPropCount

Количество пользовательских свойств.

Синтаксис:

UserPropCount;

Тип данных:

Number

UserProperty

Значения свойства по его имени или индексу.

Синтаксис:

UserProperty;

Тип данных:

Array.<Object>

UserPropertyName

Названия свойства

Синтаксис:

UserPropertyName;

Тип данных:

Array.<String>

Пример:

```
p = AddFrontPanel(50, 50, 100, 100, 50);
alert(p.GMax.x+' '+p.GMax.y+' '+p.GMax.z);
alert(p.GMin.x+' '+p.GMin.y+' '+p.GMin.z);
alert(p.GSize.x+' '+p.GSize.y+' '+p.GSize.z);
alert(p.GabMax.x+' '+p.GabMax.y+' '+p.GabMax.z);
alert(p.GabMin.x+' '+p.GabMin.y+' '+p.GabMin.z);
alert(p.GSize.x+' '+p.GSize.y+' '+p.GSize.z);
```

Методы

SetDefaultTransform

Установить нулевые положение и ориентацию объекта.

Синтаксис:

SetDefaultTransform();

Translate

Сместить объект.

Синтаксис:

Translate(dir);

Входные параметры:

dir – Направление смещения. Задается либо как Vector, либо в виде набора координат X, Y, Z (Number).

Rotate

Повернуть вокруг заданной оси в СК родительского объекта.

Синтаксис:

Rotate(axis,angle);

Входные параметры:

axis – ось вращения, например, AxisX или направление от начала СК на точку (Vector)

angle – угол поворота, градусы.

Пример:

p = AddPanel(100, 30);

p.Rotate(NewVector(50, 50, 50), 45);

TranslateGCS

Сместить объект в глобальной системе координат.

Синтаксис:

TranslateGCS(dir);

Входные параметры:

dir – Направление смещения. Задается либо как Vector, либо в виде набора координат X, Y, Z (Number).

RotateGCS

Повернуть вокруг заданной оси в глобальной системе координат.

Синтаксис:

RotateGCS(axis,angle);

Входные параметры:

axis – ось вращения, например, AxisX или направление от начала глобальной СК на точку (Vector).

angle – угол поворота, градусы.

Пример:

```
p = AddPanel(100, 30);
p.RotateGCS(NewVector(50, 50, 50), 45);
```

RotateX

Повернуть вокруг оси X.

Синтаксис:

```
RotateX(angle);
```

Входные параметры:

angle – угол поворота, градусы.

Пример:

```
p = AddPanel(100, 30);
p.RotateX(45);
```

RotateY

Повернуть вокруг оси Y.

Синтаксис:

```
RotateY(angle);
```

Входные параметры:

angle – угол поворота, градусы.

Пример:

```
p = AddPanel(100, 30);
p.RotateY(45);
```

RotateZ

Повернуть вокруг оси Z.

Синтаксис:

```
RotateZ(angle);
```

Входные параметры:

angle – угол поворота, градусы.

Пример:

```
p = AddPanel(100, 30);
p.RotateZ(45);
```

Orient

Развернуть объект относительно двух осей.

Синтаксис:

Orient(axis1,axis2);

Входные параметры:

axis1 – направление вдоль которого будет направлена ось Z локальной системы кординат объекта .

axis2 – направление вдоль которого будет направлена ось Y локальной системы кординат объекта .



В качестве axis1 и axis2 могут быть выбраны координатные оси СК родительского объекта или направление от начала СК на точку (Vector). axis1 и axis2 не могут быть параллельны друг другу. Сначала устанавливается ось Z объекта, затем, относительно ее определяется положение оси Y.

Пример:

e = AddPanel(100, 50);

Ax2 = NewVector(1, 20, 40);

e.Orient(AxisY, Ax2);

OrientGCS

Развернуть объект относительно двух осей в глобальной СК.

Синтаксис:

OrientGCS(axis1,axis2);

Входные параметры:

axis1 – направление вдоль которого будет направлена ось Z локальной системы кординат объекта.

axis2 – направление вдоль которого будет направлена ось Y локальной системы кординат объекта.



В качестве axis1 и axis2 могут быть выбраны координатные оси глобальной СК, например, AxisX или направление от начала глобальной СК на точку (Vector). axis1 и axis2 не могут быть параллельны друг другу. Сначала устанавливается ось Z объекта, затем, относительно ее определяется положение оси Y.

Пример:

p = AddFrontPanel(0, 0, 50, 40, 0);

y = AddFrontPanel(0, 0, 50, 40, 0);

Ax1 = NewVector(30, 60, 30);

Ax2 = NewVector(45, 45, 45);

y.Orient(Ax1, Ax2);

Reflect

Отразить объект относительно плоскости.

Синтаксис:

Reflect(pos,normal);

Входные данные:

pos – точка, через которую проходит плоскость, Vector.

normal – точка, через которую проходит перпендикуляр к плоскости, Vector.

Пример:

```
y = AddFrontPanel(0, 0, 50, 40, 0);
Ax1 = NewVector(100, 60, 30);
Ax2 = NewVector(45, 45, 45);
y.Reflect(Ax1, Ax2);
```

ToObject

Преобразовать координаты точки из глобальной СК в ЛСК объекта.

Синтаксис:

ToObject(pos);

Входные данные:

pos – точка, Vector.



Используется для преобразования координат. Например, точка начала контура панели задается методом GetPoint. Ее координаты получаются в глобальной СК. Контур строится на пласти панели. Координаты его точек – в ЛСК панели. Чтобы начать построение контура от указанной точки, необходимо перевести ее координаты в ЛСК панели.

Пример:

```
Pos = Model.Selected.ToObject(GetPoint("Укажите точку"));
alert(Pos.x + ' ' + Pos.y + ' ' + Pos.z);
```

ToGlobal

Преобразовать координаты точки из СК объекта в глобальную СК.

Синтаксис:

ToGlobal(pos);

Входные данные:

pos – точка, Vector.

Пример:

```
Pos = Model.Selected.ToGlobal(NewVector(0, 0, 0));
alert(Pos.x + ' ' + Pos.y + ' ' + Pos.z);
```

NToObject

Преобразовать нормаль из глобальной СК в ЛСК объекта.

Синтаксис:

```
NToObject(dir);
```

Входные данные:

dir – точка, Vector.

NToGlobal

Преобразовать нормаль из СК объекта в глобальную СК.

Синтаксис:

```
NToGlobal(dir);
```

Входные данные:

dir – точка, Vector.

Build

Перестроить объект после изменения его свойств.

Синтаксис:

```
Build;
```

IsOwner

Определить, является ли объект obj родителем объекта

Синтаксис:

```
IsOwner;
```

Входные параметры:

obj – предполагаемый родитель объекта, Object3.

Тип данных:

boolean

List3D

Список трехмерных объектов.

Является наследником Object3.

Является родительским объектом для Block, Assembly, Model.

Свойства:

Count – Количество элементов списка.

IsElastic – Является ли объект эластичным.

Objects – Список объектов.

FurnType – Тип блока.

SalonType – Тип для Салона.

Методы:

Load() – Добавить в список модель из файла.

ElasticResize – Растянуть объект до требуемых размеров.

Свойства:

Count

Количество элементов списка.

Синтаксис:

Count;

Выходные параметры:

Количество элементов списка.

Пример:

```
Block = AddBlock("Блок");
Block.Load('Файлы\\СлонБ.fr3d');
Block.Load('Файлы\\СлонЧ.fr3d');
n = Block.Count;
alert ('Количество элементов блока = '+n);
```

IsElastic

Являетсяли объект эластичным.

Синтаксис:

IsElastic();

Выходные параметры:

Boolean

Objects

Список объектов.

Синтаксис:

Objects;

Тип данных:

Array.Object3

Пример:

```
block = AddBlock("Блок");
block.Load('Файлы\\22.b3d');
block.Load('Файлы\\33.b3d');
block.Load('Файлы\\44.b3d');
k = block.Objects[1];
alert (k.Name);
```

FurnType

Тип блока

Синтаксис:

FurnType;

Тип данных:

String

SalonType

Тип для Салона

Синтаксис:

SalonType;

Тип данных:

SalonAnimationType

Методы:

ElasticResize

Растянуть объект до требуемых размеров.

Синтаксис:

ElasticResize(newSize);

Входные параметры:

newSize – новый размер объекта. Задается как Vector.

Load

Добавить в список модель из файла.

Синтаксис:

Load (‘имя файла’);

Входные параметры:

Относительное имя файла.

Выходные параметры:

Элемент списка.

Пример:

```
block = AddBlock("Блок");
block.Load('Файлы\\Тумба.b3d');
block.Build();
```

Panel

Панель.

Объект является наследником объектов Object3.

Создается командами AddFrontPanel, AddHorizPanel, AddVertPanel, AddPanel.

Свойства:

Contour – контур.

ContourHeight – высота контура.

ContourWidth – ширина контура.

Thickness – толщина.

Butts – нанесенные кромки панели.

Cuts – пазы.

Plastics – облицовки пласти.

MaterialName – наименование материала.

MaterialWidth – максимальная ширина материала панели.

Методы:

AddButt – добавить кромку.

AddCut – добавить паз.

AddPlastic – добавить облицовку пласти.

IsButtVisible – закрыта ли кромка другими панелями.

Свойства

Contour

Контур панели.

Синтаксис:

Contour;

Тип данных:

Contour2D

Пример:

Butts

Кромки панели.

Синтаксис:

Butts;

Тип данных:

PanelButts

Пример:

Cuts

Пазы панели.

Синтаксис:

Cuts;

Тип данных:

PanelCuts

Пример:

Plastics

Облицовки пласти панели.

Синтаксис:

Plastics;

Тип данных:

PanelPlastics

Пример:

ContourHeight

Высота контура панели.

Синтаксис:

ContourHeight;

Тип данных:

Number

Пример:

ContourWidth

Ширина контура панели.

Синтаксис:

ContourWidth;

Тип данных:

Number

Пример:

Thickness

Толщина панели.

Синтаксис:

Thickness;

Тип данных:

Number

Пример:

MaterialName

Наименование материала панели.

Синтаксис:

MaterialName;

Тип данных:

String

Пример:

MaterialWidth

Ширина материала панели.

Синтаксис:

MaterialWidth;

Тип данных:

Number

Пример:

Пример:

```
panMat = NewMaterialInput("Материал")
butMat = NewButtMaterialInput("Кромка");
plastMat = NewMaterialInput("Пластик");
Btn = NewButtonInput("OK");
p1 = Vector(100, 100, 16);
p2 = Vector(300, 300, 16);
$input();
true;
function $input(id) {
```

```
  DeleteNewObjects();
  Pan = AddFrontPanel(100, 100, 200, 200, 20);
  Pan.AddButt(butMat, 1);
  Pan.AddButt(butMat, 3);
  Cut = Pan.AddCut('Паз');
  Cut.Contour.AddCircle(10, 10, 10);
  Cut.Trajectory.AddLine(p1, p2);
  Pan.AddPlastic(plastMat, true);
  Pan.ArtPos = 'Артикул';
  Pan.Build();
  height = Pan.ContourHeight;
  width = Pan.ContourWidth;
  MW = Pan.MaterialWidth;
  if (id == Btn.id)
  {
  alert ('Высота панели= '+height+', Ширина панели= '+width);
  alert(Pan.MaterialName);
  alert(MW);
  alert(Pan.ArtPos);
   alert('Расстояние от начала координат панели до вершины габаритного параллелепипеда '+Pan.GMax.x+' '+Pan.GMax.y+' '+Pan.GMax.z);
  alert('Расстояние от глобального начала координат до вершины габаритного параллелепипеда '+Pan.GabMax.x+' '+Pan.GabMax.y+' '+Pan.GabMax.z);
  alert(Pan.GSize.x+' '+Pan.GSize.y+' '+Pan.GSize.z);
  return true;
  }
}
```

Методы

AddButt

Добавить кромку на элемент контура.

Синтаксис:

AddButt (material, elem);

Входные параметры:

material – материал кромки,

elem – индекс элемента контура, начиная с 0, против часовой стрелки от начала координат.

Выходные параметры:

PanelButt

Пример:

AddCut

Добавить паз.

Синтаксис:

AddCut(name);

Входные параметры:

name – имя паза.

Выходные параметры:

PanelCut

Пример:

AddPlastic

Добавить облицовку пласти.

Синтаксис:

AddPlastic(material, true);

Входные параметры:

material – имя облицовки,

true – сторона панели.

Выходные параметры:

PanelPlastic

Пример:

IsButtVisible

Закрыта ли кромка другими панелями?

Синтаксис:

IsButtVisible(index, dist);

Входные параметры:

index – индекс кромки,

dist – условие закрытости.

Выходные параметры:

Boolean



Кромка считается незакрытой, если на расстоянии, превышающем dist от нее отсутствуют панели.

Extrusion

Тело выдавливания.

Представляет собой тело, образованное перемещением контура профиля вдоль прямой линии. Контур профиля должен быть замкнутым.

Объект является наследником объекта Object3.

Создается командой AddExtrusion.

Свойства:

Contour – контур профиля,

Thickness – толщина тела выдавливания,

MaterialName – наименование материала.

Свойства

Contour

Контур профиля тела выдавливания. Контур должен быть замкнутым.

Синтаксис:

Contour;

Тип данных:

Contour2D

Пример:

Thickness

Толщина тела выдавливания.

Синтаксис:

Thickness;

Тип данных:

Number

Пример:

MaterialName

Наименование материала тела выдавливания.

Синтаксис:

MaterialName;

Тип данных:

String

Пример:

Пример:

```
Extr = AddExtrusion('Бобышка');
Extr.Contour.AddCircle(100, 100, 50);
Extr.Thickness = 50;
Extr.Build();
ExtrName = Extr.MaterialName;
alert (ExtrName);
```

Trajectory

Траектория представляет собой кривую, вдоль которой тело, образованное перемещением контура профиля вдоль траектории. Контур траектории может быть замкнутым или разомкнутым. Контур профиля должен быть замкнутым.

Объект является наследником объекта Object3.

Создается командой AddTrajectory.

Свойства:

Contour2D – контур профиля,

Trajectory2D – контур траектории,

MaterialName – наименование материала.

Свойства

Contour2D

Контур профиля кинематического тела. Контур должен быть замкнутым.

Синтаксис:

Contour2D;

Тип данных:

Contour2D

Пример:

Trajectory2D

Контур траектории кинематического тела.

Синтаксис:

Trajectory2D;

Тип данных:

Contour2D

Пример:

MaterialName

Наименование материала кинематического тела .

Синтаксис:

MaterialName;

Тип данных:

String

Пример:

Пример:

```
Traj = AddTrajectory('Траектория');
p1 = NewVector(0, 0, 0);
p2 = NewVector(40, 50, 70);
p3 = NewVector(100, 150, 120);
Traj.Trajectory2D.AddArc3(p1,  p2,  p3);
Traj.Contour2D.AddRoundRect(30, 50, 50, 70, 8);
TrName = Traj.MaterialName;
alert (TrName);
```

Block

Блок. Представляет собой именованую группу элементов модели. Создается командой AddBlock(). Доступ к элементам блока можно осуществить по индексу. Нумерация начинается с 0.

Объект является наследником объекта List3D.

Свойства:

IsFastener – Флаг составной фурнитуры.

Свойства:

IsFastener

Флаг составной фурнитуры

Синтаксис:

IsFastener;

Тип данных:

Boolean

Assembly

Сборка. Представляет собой именованую группу элементов модели. Создается командой AddAssembly(). Доступ к элементам сборки можно осуществить по индексу. Нумерация начинается с 0.

Объект является наследником объекта List3D.

Свойства:

AnimType – Тип анимации.

Свойства

AnimType

Тип анимации сборки.

Синтаксис:

AnimType;

Тип данных:

AnimationType

Пример:

```
asm = AddAssembly("Сборка");
p = AddFrontPanel(0, 0, 40, 40, 0);
p.Owner = asm;
asm.AnimType = AnimationType.DoorRight;
```

PanelButts

Кромки.

Создаются командой Butts.

Свойства:

Count – количество кромок.

Методы:

Butts[] – получить кромку панели по индексу.

Add – добавить кромку.

Свойства

Count

Количество кромок.

Синтаксис:

Count;

Тип данных:

Number

Пример:

```
Pan = AddFrontPanel(0, 0, 400, 400, 0);
  Pan.AddButt(butMat, 1);
  Pan.AddButt(butMat, 3);
  Pan.AddPlastic(plastMat, true);
  Pan.Build();
  height = Pan.ContourHeight;
  width = Pan.ContourWidth;
  butts = Pan.Butts;
```

Методы

Butts[]

Получить кромку панели по индексу.

Синтаксис:

Butts[i];

Входные параметры:

i – индекс кромки панели, начиная с 0.

Тип данных:

PanelButt

Add

Добавить кромку панели.

Синтаксис:

Add;

Выходные параметры:

PanelButt



Рекомендуется применять эту команду только в случае добавления каких-либо специфических кромок. В результате выполнения команды создается «пустая» кромка, параметры которой необходимо задавать при помощи дополнительных команд. В большинстве случаев целесообразно использовать команду AddButt.

PanelButt

Кромка панели.

Свойства:

ElemIndex – Индекс кромки.

Sign – Обозначение кромки.

Material – Материал кромки.

Thickness – Толщина кромки.

Width – Ширина кромки.

ClipPanel – Подрезать панель на толщину кромки.

Overhung – Свес, на сколько лента кромки должна быть длиннее торца детали.

Allowance – Припуск, на сколько прифрезеровать торец перед кромкованием.

CutIndex – Номер отреза (для наклейки кромки на несколько торцев одним отрезом).

Profile – Профиль кромки.

Свойства

ElemIndex

Индекс кромки.

Синтаксис:

ElemIndex;

Тип данных:

Number

Sign

Обозначение кромки.

Синтаксис:

Sign;

Тип данных:

String

Material

Материал кромки.

Синтаксис:

Material;

Тип данных:

String

Thickness

Толщина кромки.

Синтаксис:

Thickness;

Тип данных:

Number

Width

Ширина кромки.

Синтаксис:

Width;

Тип данных:

Number

ClipPanel

Подрезать панель на толщину кромки.

Синтаксис:

ClipPanel;

Тип данных:

boolean

Значения свойства:

True – подрезать

False – не подрезать

Overhung

Свес, на сколько лента кромки должна быть длиннее торца детали.

Синтаксис:

Overhung;

Тип данных:

Number

Allowance

Припуск - на сколько прифрезеровать торец перед кромкованием.

Синтаксис:

Allowance;

Тип данных:

Number

CutIndex

Номер отреза (для наклейки кромки на несколько торцев одним отрезом).

Синтаксис:

CutIndex;

Тип данных:

Number

Profile

Профиль кромки.

Синтаксис:

Profile;

Тип данных:

Contour2D

PanelPlastics

Создаются командой Plastics.

Свойства:

Count – количество облицовок пласти.

Методы:

Plastics[] – получить облицовку пласти по индексу.

Add – добавить облицовку.

Свойства

Count

Количество облицовок пласти.

Синтаксис:

Count;

Тип данных:

Number

Пример:

```
Pan = AddFrontPanel(0, 0, 400, 400, 0);
  Pan.AddButt(butMat, 1);
  Pan.AddButt(butMat, 3);
  Pan.AddPlastic(plastMat, true);
  Pan.Build();
  height = Pan.ContourHeight;
  width = Pan.ContourWidth;
  butts = Pan.Butts;
```

Методы

Plastics[]

Получить облицовку по индексу.

Синтаксис:

Plastics[i];

Входные параметры:

i – индекс облицовки, начиная с 0.

Выходные параметры:

PanelPlastic

Add

Добавить облицовку.

Синтаксис:

Add();

Выходные параметры:

PanelPlastic



Рекомендуется применять эту команду только в случае добавления каких-либо специфических облицовок пласти. В результате выполнения команды создается «пустая» облицовка, параметры которой необходимо задавать при помощи дополнительных команд. В большинстве случаев целесообразно использовать команду AddPlastic.

PanelPlastic

Пластик панели.

Свойства:

Material – Материал пластика.

Thickness – Толщина пластика.

TextureOrientation – Ориентация текстуры пластика.

Свойства:

Material

Материал пластика панели.

Синтаксис:

Material;

Тип данных:

String

Thickness

Толщина пластика панели.

Синтаксис:

Thickness;

Тип данных:

Number

TextureOrientation

Ориентация текстуры пластика панели.

Синтаксис:

TextureOrientation;

Тип данных:

TextureDir

PanelCuts

Пазы панели.

Свойства:

Count – количество пазов.

Методы:

Cuts[] – получить паз по индексу.

Add – добавить паз.

Свойства

Count

Количество пазов.

Синтаксис:

Count;

Тип данных:

Number

Пример:

```
 Pan = AddFrontPanel(0, 0, 400, 400, 0);
   Pan.AddButt(butMat, 1);
   Pan.AddButt(butMat, 3);
   Pan.AddPlastic(plastMat, true);
   Pan.Build();
   height = Pan.ContourHeight;
```

```
width = Pan.ContourWidth;
butts = Pan.Butts;
```

Методы

Add

Добавить паз.

Синтаксис:

Add();

Выходные параметры:

PanelCut

Cuts

Получить паз по индексу.

Синтаксис:

Cuts[i];

Входные параметры:

i – индекс паза, начиная с 0.

Выходные параметры:

PanelCut

PanelCut

Паз.

Свойства:

Name – Имя паза.

Sign – Обозначение паза.

Trajectory – Траектория паза.

Contour – Профиль паза.

Свойства

Name

Имя паза.

Синтаксис:

Name;

Тип данных:

String

Sign

Обозначение паза.

Синтаксис:

Sign;

Тип даных:

String

Trajectory

Траектория паза.

Синтаксис:

Trajectory;

Тип данных:

Contour2D

Contour

Профиль паза.

Синтаксис:

Contour;

Тип данных:

Contour2D

Contour2D

Контур.

Свойства:

Count – Количество элементов контура.

Width – Ширина контура.

Height – Высота контура.

Min – Левый нижний угол охватывающего прямоугольника.

Max – Правый верхний угол охватывающего прямоугольника.

Методы:

Clear –Очистить контур.

Move – Сдвинуть все элементы контура.

Rotate – Повернуть контур вокруг точки.

AddRectangle – Добавить прямоугольник.

AddRoundRect – Добавить прямоугольник со скругленными краями.

AddLine – Добавить линию.

AddCircle – Добавить окружность.

AddArc – Добавить дугу по начальной, конечной точкам и центру.

AddArc3 – Добавить дугу по 3 точкам.

AddEquidistantRecursive – Добавить эквидистанту контура (включая вложенные контуры).

Subtraction – Вычесть замкнутый контур.

Addition – Сложить с замкнутым контуром.

RoundingEx – Построить скругление в точке пересечения двух отрезков контура.

FacetEx – Построить фаску в точке пересечения двух отрезков контура.

Rounding – Построить скругление в точке пересечения двух элементов контура.

Facet – Построить фаску в точке пересечения двух отрезков контура.

Find – Найти ближайший элемент по координатам.

Fit – Вписать весь контур в заданные габариты.

Elastic – Изменить контур резиновой нитью.

Symmetry – Создать копию объекта, симметричную относительно линии.

Load – Загрузить контур из файла фрагмента.

OrderContours – Упорядочить элеметны контура в одном направлении.

AddEquidistant – Добавить эквидистанту контура.

IsPointInside – Находится ли точка внутри контура?

IsInContour – Находится ли контур внутри другого?

IsClosedContour – Является ли контур замкнутым?

IsContourRectangle – Является ли контур прямоугольным?

Пример:

```
Obj = AddPanel(300, 300);
Pan = Obj.AsPanel; //является ли выделенный объект панелью
if (Pan) {
  c = Pan.Contour;
  Undo.Changing(Obj); // укажем, что этот объект будет редактироваться
  //p = Action.GetPoint('Укажите центр отверстия');
  p = NewVector(40, 40, 40);
  p = Obj.ToObject(p); // переведем точку в систему координат контура панели
  Hole = NewContour();
  Hole.AddRectangle(40, 40, 100, 110);
  //Hole.Rounding(59, 45, 30);
```

```
  Hole.Facet(80, 100, 10);
 // Hole.Facet(80, 100, 40);
  Hole.Rotate(40,40,30);
  c.Substraction(Hole);
      ro = NewContour();
      e1 = ro.AddLine(100, 160, 190, 190);
      e2 = ro.AddLine(160, 60, 190, 190);
      e3 = ro.AddLine(100, 160, 160, 60);
      ro.RoundingEx(e1, e2, 170, 170, 20);
        f = ro.Find(110, 140);
      ro.RoundingEx(e2, f, 150, 90, 26,5);
      ro.FacetEx(e1, e3, 20, 20);
    ro.Fit(100, 30, 170, 200);
    ro.Elastic(100, 30, 170, 60, 50, 70);
    ro.Symmetry(185, 30, 150, 200, false);
    c.Substraction(ro);
        lo = NewContour();
        lo.Load('contour.frw');
        lo.Move(55, 180);
        c.Substraction(lo);
  Obj.Build(); // перестроим объект после всех изменений
}
```

Свойства

Count

Количество элементов контура.

Синтаксис:

Count;

Тип данных:

Number

Пример:

Objects

Список элементов контура.

Синтаксис:

Objects;

Тип данных:

Array.Elem2D

Width

Ширина контура.

Синтаксис:

Width;

Тип данных:

Number

Пример:

Height

Высота контура.

Синтаксис:

Height;

Тип данных:

Number

Пример:

Min

Левый нижний угол охватывающего прямоугольника.

Синтаксис:

Min;

Возвращаемое значение:

Point

Max

Правый верхний угол охватывающего прямоугольника.

Синтаксис:

Max;

Возвращаемое значение:

Point

Методы

Clear

Очистить контур.

Синтаксис:

Clear();

Пример:

Move

Сдвинуть все элементы контура.

Синтаксис:

Move(dx,dy);

Входные параметры:

dx, dy – значения смещений по соответствующим осям.

Пример:

Rotate

Повернуть контур вокруг точки.

Синтаксис:

Rotate(x, y, angle);

Входные параметры:

x, y – координаты центра вращения.

angle – угол поворота

AddRectangle

Добавить прямоугольник.

Синтаксис:

AddRectangle(x1, y1, x2, y2);

Входные параметры:

x1, y1, x2, y2 – координаты начальной и конечной точек диагонали прямоугольника.

Пример:

AddRoundRect

Добавить прямоугольник со скругленными вершинами.

Синтаксис:

AddRoundRect(x1, y1, x2, y2, rad);

Входные параметры:

x1, y1, x2, y2 – координаты начальной и конечной точек диагонали прямоугольника.

rad – радиус скругления.

Пример:

AddLine

Добавить линию.

Синтаксис:

AddLine(x1, y1, x2, y2);

Входные параметры:

x1, y1, x2, y2 – координаты начальной и конечной точек линии.

Пример:

AddCircle

Добавить окружность.

Синтаксис:

AddCircle(xc, yc, rad);

Входные параметры:

xc, yc – координаты центра окружности.

rad – радиус окружности.

Пример:

AddArc

Добавить дугу по начальной, конечной точкам и центру.

Синтаксис:

AddArc(p1, p2, centre, dir);

Входные параметры:

p1, p2 – начальная и конечная точки (Vector).

centre – центр дуги (Vector).

dir –направление дуги:

- true – от p1 к p2;
- false – от p2 к p1;

Пример:

AddArc3

Добавить дугу по трем точкам.

Синтаксис:

AddArc3(p1, p2, p3);

Входные параметры:

p1, p2, p3 – точки дуги (Vector).

Пример:

AddEquidistantRecursive

Добавить эквидистанту контура (включая вложенные контуры). Последние 2 параметра отвечают за направление и скругление.

Синтаксис:

AddEquidistantRecursive(contour, offset, Side, Rounding);

Входные параметры:

p1, p2, p3 – точки дуги (Vector).

contour – контур, Contour2D

offset – смещение, number

Side – направление смещения, boolean

Rounding – скругление вершин, boolean

Subtraction

Вычесть замкнутый контур.

Синтаксис:

Subtraction(contour);

Входные параметры:

contour – вычитаемый контур (Contour2D).

Пример:

Addition

Добавить замкнутый контур.

Синтаксис:

Addition(contour);

Входные параметры:

contour – добавляемый контур (Contour2D).

Пример:

RoundingEx

Построить скругление в точке пересечения двух отрезков контура.

Синтаксис:

RoundingEx(elem1, elem2, x, y, radius);

Входные параметры:

elem1, elem2 – отрезки контура, между которыми строится скругление,

x, y – координаты точки, определяющей выбор скругляющей дуги, в СК контура.

radius – радиус скругления.



Метод целесообразно использовать, если контур состоит из отдельных именованых отрезков. Для скругления вершин прямоугольника следует использовать метод Rounding.

Пример:

FacetEx

Построить фаску в точке пересечения двух отрезков контура.

Синтаксис:

FacetEx(elem1, elem2, l1, l2);

Входные параметры:

elem1, elem2 – элементы контура, между которыми строится фаска,

l1, l2 – отступы от вершины для соответствующих элементов.

Пример:

Rounding

Построить скругление в точке пересечения двух элементов контура.

Синтаксис:

Rounding(x, y, radius);

x, y – координаты точки, определяющей выбор скругляющей дуги в СК контура.

radius – радиус скругления.



Метод целесообразно использовать для скругления вершин прямоугольника. Если контур состоит из отдельных именованых отрезков, следует использовать метод RoundingEx.

Пример:

Facet

Построить фаску в точке пересечения двух элементов контура.

Синтаксис:

Facet(x, y, l);

Входные параметры:

x, y – координаты точки, определяющей выбор вершины построения фаски в СК контура.

l – длина фаски (одинакова для обоих элементов).

Пример:

Find

Найти элемент, ближайший к точке с заданными координатами.

Синтаксис:

Find(x, y);

Входные параметры:

x, y – координаты точки.

Пример:

Fit

Вписать контур в заданные габариты.

Синтаксис:

Fit(x1, y1, x2, y2);

Входные параметры:

x1, y1, x2, y2 – координаты концов диагонали габаритного прямоугольника.

Пример:

Elastic

Изменить контур резиновой нитью.

Синтаксис:

Elastic(x1, y1, x2, y2, dx, dy);

Входные параметры:

x1, y1, x2, y2 – координаты концов диагонали габаритного прямоугольника, охватывающего часть контура.

dx, dy – направление смещения.

Пример:

Symmetry

Создать копию объекта, симметричную относительно линии.

Синтаксис:

Symmetry(x1, y1, x2, y2, del);

Входные параметры:

x1, y1, x2, y2 – координаты концов оси симметрии.

del – признак удаления исходного объекта:

- true – не удалять;
- false – удалять.

Пример:

Load

Загрузить контур из файла фрагмента.

Синтаксис:

Load(“file”);

Входные параметры:

file – имя файла фрагмента с расширением frw.

Пример:

OrderContours

Упорядочить элементы контура в одном направлении.

Синтаксис:

OrderContours;

Возвращаемое значение:

Boolean

AddEquidistant

Добавить эквидистанту контура.

Синтаксис:

AddEquidistant(contour,offset,false,false);

Входные параметры:

contour – указатель на исходный контур.

offset – смещение.

dir – флаг направления.

fillet – флаг скругления.

Пример:

```
X=NewContour();
X.Load(ttt.frw);
Y=NewContour();
Y.AddEquidistant(X,5, false,true);
```

IsPointInside

Проверить, находится ли точка внутри контура?

Синтаксис:

IsPointInside(x,y);

Входные параметры:

x, y – координаты точки.

Возвращаемое значение:

Boolean

IsInContour

Проверить, находится ли контур внутри другого.

Синтаксис:

IsInContour(contour);

Входные параметры:

contour – контур, нахождение внутри которого проверяется.

Возвращаемое значение:

Boolean

IIsClosedContour

Проверить, является ли контур замкнутым.

Синтаксис:

IsClosedContour;

Возвращаемое значение:

Boolean

IsContourRectangle

Проверить, является ли контур прямоугольным.

Синтаксис:

IsContourRectangle;

Возвращаемое значение:

Boolean

InControl

Элемент управления.

Свойства:

id – идентификатор.

Enabled – доступность объекта.

Visible – видимость объекта.

Hint – текст отображаемой подсказки.

OnChange – функция-обработчик.

Свойства

id

Идентификатор.

Синтаксис:

id;

Тип данных:

Number

Enabled

Доступность объекта.

Синтаксис:

Enabled;

Тип данных:

Boolean

Visible

Видимость объекта.

Синтаксис:

Visible;

Тип данных:

Boolean

Hint

Текст отображаемой подсказки.

Синтаксис:

Hint;

Тип данных:

String

OnChange

Функция-обработчик.

Синтаксис:

OnChange;

Тип данных:

Function

InButton

Кнопка.

Свойства:

Caption – Текст на кнопке.

Combo – Отдельная стрелка для подменю.

Методы:

NewSubMenu – Создать подменю.

Свойства:

Caption

Текст на кнопке.

Синтаксис:

Caption;

Тип данных:

String

Combo

Отдельная стрелка для подменю**.**

Синтаксис:

Combo;

Тип данных:

Boolean

Методы:

NewSubMenu

Создать подменю.

Синтаксис:

NewSubMenu(MenuItemName);

Входные данные:

MenuItemName – Имя пункта меню (String).

Тип данных:

InButton

InFloat

Число с плавающей точкой.

Свойства:

Value – Значение.

ReadOnly – Только для чтения.

Fixed – Значение задано в поле ввода.

Свойства

Value

Значение.

Синтаксис:

Value;

Тип данных:

Number

ReadOnly

Только для чтения.

Синтаксис:

ReadOnly;

Тип данных:

Boolean

Fixed

Значение задано в поле ввода.

Синтаксис:

Fixed;

Тип данных:

Boolean

InNumber

Свойства:

Value – Значение.

ReadOnly – Только для чтения.

Fixed – Значение задано в поле ввода.

Свойства

Value

Значение.

Синтаксис:

Value;

Тип данных:

Number

ReadOnly

Только для чтения.

Синтаксис:

ReadOnly;

Тип данных:

Boolean

Fixed

Значение задано в поле ввода.

Синтаксис:

Fixed;

Тип данных:

Boolean

InMaterial

Материал

Свойства:

Name – наименование материала.

Thickness – толщина материала.

Width – ширина материала.

Методы:

SetActive – установить активным.

Apply – применить материал к указанному объекту.

Свойства

Name

Наименование материала.

Синтаксис:

Name;

Тип данных:

String

Thickness

Толщина материала.

Синтаксис:

Thickness;

Тип данных:

Number

Width

Ширина материала.

Синтаксис:

Width;

Тип данных:

Number

Методы

SetActive

Установить материал активным. Все последующие элементы будут построены из этого материала.

Синтаксис:

SetActive();

Apply

Применить материал к указанному объекту.

Синтаксис:

Apply(obj);

Входные данные:

obj – объект применения материала (Object3).

Тип данных:

Object3

InButtMaterial

Материал кромки.

Свойства:

Name – наименование материала.

Sign – обозначение материала.

Thickness – толщина материала.

Width – ширина материала.

Overhung – свес.

Allowance – припуск.

ClipPanel – подрезка панели.

Свойства

Name

Наименование материала.

Синтаксис:

Name;

Тип данных:

String

Sign

Обозначение материала.

Синтаксис:

Sign;

Тип данных:

String

Thickness

Толщина материала.

Синтаксис:

Thickness;

Тип данных:

Number

Width

Ширина материала.

Синтаксис:

Width;

Тип данных:

Number

Overhung

Свес.

Синтаксис:

Overhung;

Тип данных:

Number

Allowance

Припуск.

Синтаксис:

Allowance;

Тип данных:

Number

ClipPanel

Подрезка панели.

Синтаксис:

ClipPanel;

Тип данных:

Boolean

InFurniture

Фурнитура.

Свойства:

DatumModeFilter – Фильтр по типу монтирования фурнитуры/фрагмента.

DatumMode – Тип монтирования выбранной фурнитуры/фрагмента.

Свойства

DatumModeFilter

Фильтр по типу монтирования фурнитуры/фрагмента.

Синтаксис:

DatumModeFilter;

Тип данных:

DatumMode

DatumMode

Тип монтирования выбранной фурнитуры/фрагмента.

Синтаксис:

DatumMode;

Тип данных:

DatumMode

Методы:

Mount – Установить крепеж между двух панелей.

Mount1 – Установить крепеж на плоскость панели.

Методы

Mount

Установить крепеж между двух панелей.

Синтаксис:

Mount(panel1,panel2,x,y,z);

Входные данные:

panel1,panel2 – соединяемые панели, Object3.

x, y, z – координаты точки установки фурнитуры; могут быть заданы как Vector.

Mount1

Установить крепеж на плоскость панели.

Синтаксис:

Mount1(panel, x, y, z, angle);

Входные параметры:

panel – панель, на которую устанавливается фурнитура, Object3.

x, y, z – координаты точки установки фурнитуры, number, могут быть заданы как Vector.

angle – угол поворота фурнитуры вокруг горизонтальной оси в градусах, number.

MountScheme

Установить схему крепежа на стык панелей.

Синтаксис:

MountScheme (panel1, panel2, FurnPos, basisPoint);

Входные параметры:

panel1, panel2 – панели, имеющие стык, Panel.

FurnPos – Позиция фурнитуры при установке крепежа, FurniturePosition.

basisPoint – базовая точка установки фурнитуры; может быть задана как Vector.

MountBox

Установить секцию.

Синтаксис:

MountBox (Position,Size,AxisZ,AxisY

Входные параметры:

Position – Позиция секции, Vector.

Size – Размер секции, Vector.

AxisZ – Направление оси Z секции, Vector.

AxisY – Направление оси Y секции, Vector.

ModelInspector

Анализ модели

Свойства:

ErrorList – Список ошибок.

Options – Опции анализа.

Методы:

Run – Проверить модель.

Свойства

ErrorList

Список ошибок.

Синтаксис:

ErrorList;

Тип данных:

Array <InspectorError>

Options

Опции анализа.

Синтаксис:

Options;

Тип данных:

InspectorOptions

Методы

Run

Проверить модель.

Синтаксис:

Run(Model);

Входные данные:

Model – проверяемая модель (List3D).

Возвращаемое значение:

Отсутствует

InspectorOptions

Параметры анализа модели

Свойства:

ObjIntersectionAnalyze – Проверка пересечения объектов.

FastIntersectionAnalyze – Проверка пересечения фурнитуры.

FastIncorrectAnalyze – Проверка корректности фурнитуры.

PanelNotFixedAnalyze – Проверка скрепления панелей.

PanelTooLargeAnalyze – Проверка размера панели на плите.

PlasticTooLargeAnalyze – Проверка размера пластика на панели.

MatNotExistsAnalyze – Проверка материала в наличии.

MatOutOfStockAnalyze – Проверка материала на складе.

Свойства

ObjIntersectionAnalyze

Проверка пересечения объектов.

Синтаксис:

ObjIntersectionAnalyze;

Тип данных:

boolean

FastIntersectionAnalyze

Проверка пересечения фурнитуры.

Синтаксис:

FastIntersectionAnalyze;

Тип данных:

boolean

FastIncorrectAnalyze

Проверка пересечения фурнитуры.

Синтаксис:

FastIncorrectAnalyze;

Тип данных:

boolean

PanelNotFixedAnalyze

Проверка скрепления панелей.

Синтаксис:

PanelNotFixedAnalyze;

Тип данных:

boolean

PanelTooLargeAnalyze

Проверка размера панели на плите.

Синтаксис:

PanelTooLargeAnalyze;

Тип данных:

boolean

PlasticTooLargeAnalyze

Проверка размера пластика на панели.

Синтаксис:

PlasticTooLargeAnalyze;

Тип данных:

boolean

MatNotExistsAnalyze

Проверка материала в наличии.

Синтаксис:

MatNotExistsAnalyze;

Тип данных:

boolean

MatOutOfStockAnalyze

Проверка материала на складе.

Синтаксис:

MatOutOfStockAnalyze;

Тип данных:

boolean

InspectorError

Параметры найденной ошибки

Свойства:

ErrorType – Тип ошибки.

ErrorObjectsCount – Количество объектов, относящихся к ошибке.

ErrorObjects – Список объектов, относящихся к ошибке.

ErrorMessage – Сообщение ошибки.

ObjectsNames – Имена объектов в ошибке.

Свойства

ErrorType

Тип ошибки.

Синтаксис:

ErrorType;

Тип данных:

ErrorType

ErrorObjectsCount

Количество объектов, относящихся к ошибке.

Синтаксис:

ErrorObjectsCount;

Тип данных:

number

ErrorObjects

Список объектов, относящихся к ошибке.

Синтаксис:

ErrorObjects;

Тип данных:

Array<Object3>

ErrorMessage

Сообщение ошибки.

Синтаксис:

ErrorMessage;

Тип данных:

string

ObjectsNames

Имена объектов в ошибке.

Синтаксис:

ObjectsNames;

Тип данных:

string

ImportExportSVG

Импорт-экспорт в SVG

Свойства:

CurveQuality – Качество аппроксимации кривых в линии [0-100].

GroupElems – Объединять элементы в полигоны, полилинии и кривые.

Методы:

Save – Сохранить контур.

Load – Загрузить контур.

Свойства

CurveQuality

Качество аппроксимации кривых в линии [0-100].

Синтаксис:

CurveQuality;

Тип данных:

number

GroupElems

Объединять элементы в полигоны, полилинии и кривые.

Синтаксис:

GroupElems;

Тип данных:

boolean

Методы

Save

Сохранить контур.

Синтаксис:

Save(filename, contour);

Входные данные:

filename – имя файла, string

contour – контур, данные которого сохранятся в файл, Contour2D.

Возвращаемое значение:

boolean

Load

Загрузить контур.

Синтаксис:

Load(filename, contour);

Входные данные:

filename – имя файла, string

contour – куда считаются данные из файла, Contour2D.

Возвращаемое значение:

boolean

ImportExport

Импорт-экспорт

Свойства:

SVG – Импорт-экспорт в SVG.

Свойства

SVG

Импорт-экспорт в SVG.

Синтаксис:

SVG;

Тип данных:

ImportExportSVG

InfFurniture

Информация о фурнитуре

Свойства:

DatumModeFilter – Фильтр по типу монтирования фурнитуры/фрагмента.

DatumMode – Тип монтирования выбранной фурнитуры/фрагмента.

Методы:

Mount – Установить крепеж между двух панелей.

Mount1 – Установить крепеж на плоскость панели.

MountScheme – Установить схему крепежа на стык панелей.

MountBox – Установить секцию.

Make – Создать объект фурнитуры.

EncodeToString – Зашифровать параметры фурнитуры в строку.

DecodeFromString – Восстановить параметры фурнитуры из строки.

Choose – Открыть окно выбора фурнитуры.

Свойства

DatumModeFilter

Фильтр по типу монтирования фурнитуры/фрагмента.

Синтаксис:

DatumModeFilter;

Тип данных:

DatumMode

DatumMode

Тип монтирования выбранной фурнитуры/фрагмента.

Синтаксис:

DatumMode;

Тип данных:

DatumMode

Методы

Mount

Установить крепеж между двух панелей.

Синтаксис:

Mount(panel1, panel2, x, y, z);

Входные данные:

panel1, panel2 – панели, Panel.

x, y, z – координаты, number

Возвращаемое значение:

Object3

Mount1

Установить крепеж на плоскость панели.

Синтаксис:

Mount1(panel, x, y, z, angle);

Входные данные:

panel1 – панель, Panel.

x, y, z – координаты, number

angle – угол в градусах, number

Возвращаемое значение:

Object3

MountScheme

Установить схему крепежа на стык панелей.

Синтаксис:

MountScheme(panel1, panel2, FurniturePosition, BasisPoint);

Входные данные:

panel1 – панель, Panel.

FurniturePosition – позиция фурнитуры, FurniturePosition

BasisPoint – базовая точка, Vector

Возвращаемое значение:

Object3

MountBox

Установить секцию.

Синтаксис:

MountBox(Position, Size, axisZ, axisY);

Входные данные:

Position – Позиция секции, Vector

Size – Размер секции, Vector

axisZ - Направление оси Z секции, Vector

axisY - Направление оси Y секции, Vector

Возвращаемое значение:

Object3

Make

Создать объект фурнитуры.

Синтаксис:

Make(Thickness1, Thickness2);

Входные данные:

Thickness1, Thickness2 – толщины панелей, number

Возвращаемое значение:

Object3

EncodeToString

Создать объект фурнитуры.

Синтаксис:

EncodeToString();

Возвращаемое значение:

string

DecodeFromString

Создать объект фурнитуры.

Синтаксис:

DecodeFromString(str);

Входные данные:

str – строка параметров фурнитуры, сохраненная методом EncodeToString, string

Возвращаемое значение:

нет

Choose

Открыть окно выбора фурнитуры.

Синтаксис:

Choose();

Возвращаемое значение:

boolean (true – выбор сделан, false – выбор отменен)

RootProperties

Корневые свойства

Методы:

NewFurnitureValue – Создать объект с информацией о фурнитуре.

Методы

NewFurnitureValue

Создать объект с информацией о фурнитуре.

Синтаксис:

NewFurnitureValue();

Возвращаемое значение:

InfFurniture

Elem2D

2D-элемент.

Свойства:

ElType – тип 2D элемента.

Методы:

IsLine – Является ли элемент линией.

AsLine – Получить элемент как линию.

IsCircle – Является ли элемент окружностью.

AsCircle – Получить элемент как окружность.

IsEllipse – Является ли элемент эллипсом.

AsEllipse – Получить элемент как эллипс.

IsArc – Является ли элемент дугой.

AsArc – Получить элемент как дугу.

IsList – Является ли элемент списком элементов.

AsList – Получить элемент как список элементов.

ObjLength – Длина элемента.

Свойства

ElType

Тип 2D элемента.

Синтаксис:

ElType;

Тип данных:

ElementType

Методы

IsLine

Является ли элемент линией.

Синтаксис:

IsLine();

Возвращаемое значение:

boolean

AsLine

Получить элемент как линию.

Синтаксис:

AsLine();

Возвращаемое значение:

Line2D

IsCircle

Является ли элемент окружностью.

Синтаксис:

IsCircle();

Возвращаемое значение:

boolean

AsCircle

Получить элемент как окружность.

Синтаксис:

AsCircle();

Возвращаемое значение:

Circle2D

IsEllipse

Является ли элемент эллипсом.

Синтаксис:

IsEllipse();

Возвращаемое значение:

boolean

AsEllipse

Получить элемент как эллипс.

Синтаксис:

AsEllipse();

Возвращаемое значение:

Ellipse2D

IsArc

Является ли элемент дугой.

Синтаксис:

IsArc();

Возвращаемое значение:

boolean

AsArc

Получить элемент как дугу.

Синтаксис:

AsArc();

Возвращаемое значение:

Arc2D

IsList

Является ли элемент списком элементов.

Синтаксис:

IsList();

Возвращаемое значение:

boolean

AsList

Получить элемент как список элементов.

Синтаксис:

AsList();

Возвращаемое значение:

Contour2D

ObjLength

Длина элемента.

Синтаксис:

ObjLength();

Возвращаемое значение:

number

Line2D

2D линия

Свойства:

Pos1 – Начало.

Pos2 – Конец.

Свойства

Pos1

Начало.

Синтаксис:

Pos1;

Тип данных:

Point

Pos2

Конец.

Синтаксис:

Pos2;

Тип данных:

Point

Arc2D

2D дуга

Свойства:

Pos1 – Начало.

Pos2 – Конец.

Center – Центр.

ArcDir – Направление.

Свойства

Pos1

Начало.

Синтаксис:

Pos1;

Тип данных:

Point

Pos2

Конец.

Синтаксис:

Pos2;

Тип данных:

Point

Center

Центр.

Синтаксис:

Center;

Тип данных:

Point

ArcDir

Направление.

Синтаксис:

ArcDir;

Тип данных:

boolean

Принимаемые значения:

True – против часовой стрелки;

False – по часовой стрелке;

Circle2D

2D окружность

Свойства:

Center – Центр.

CirRadius – Радиус.

Dir – Направление.

Свойства

Center

Центр.

Синтаксис:

Center;

Тип данных:

Point

CirRadius

Радиус.

Синтаксис:

CirRadius;

Тип данных:

number

Dir

Направление.

Синтаксис:

ArcDir;

Тип данных:

boolean

Принимаемые значения:

True – против часовой стрелки;

False – по часовой стрелке;

Ellipse2D

2D эллипс

Свойства:

Center – Центр.

MajorRadius – Большой радиус - имеет тип number

MinorRadius – Малый радиус - имеет тип number

MajorAxisAngle – Угол - имеет тип number

Dir – Направление

Свойства

Center

Центр.

Синтаксис:

Center;

Тип данных:

Point

MajorRadius

Большой радиус.

Синтаксис:

MajorRadius;

Тип данных:

number

MinorRadius

Малый радиус.

Синтаксис:

MinorRadius;

Тип данных:

number

MajorAxisAngle

Угол.

Синтаксис:

MajorAxisAngle;

Тип данных:

number

Dir

Направление.

Синтаксис:

Dir;

Тип данных:

boolean

Принимаемые значения:

True – против часовой стрелки;

False – по часовой стрелке;

FurnArticle

Артикул модели

Свойства:

Name – Имя модели.

OrderCode – Код заказа.

OrderName – Имя заказа.

DatumMode – Тип установки фурнитуры/фрагмента.

Методы:

NameWithCode – Получить имя и артикул.

Свойства

Name

Имя модели.

Синтаксис:

Name;

Тип данных:

string

OrderCode

Код заказа.

Синтаксис:

OrderCode;

Тип данных:

string

OrderName

Имя заказа.

Синтаксис:

OrderName;

Тип данных:

string

DatumMode

Тип установки фурнитуры/фрагмента.

Синтаксис:

DatumMode;

Тип данных:

DatumMode

Методы

NameWithCode

Получить имя и артикул.

Синтаксис:

NameWithCode();

Возвращаемое значение:

string

БАЗИС-Мебельщик

Глобальные константы БАЗИС-Мебельщик

p3dFront

Вид спереди.

Тип данных:

Number

p3dLeft

Вид слева.

Тип данных:

Number

p3dRight

Вид справа.

Тип данных:

Number

p3dTop

Вид сверху.

Тип данных:

Number

p3dBottom

Вид снизу.

Тип данных:

Number

p3dIsometric

Аксонометрия.

Тип данных:

Number

БАЗИС-Смета

Глобальные свойства БАЗИС-Смета

Panel

Доступ к параметрам панели.

Синтаксис:

Panel;

Тип данных:

Panel

Holes

Список отверстий панели

Синтаксис:

Holes;

Тип данных:

PanelHoles

Holes: свойство типа PanelHoles - 'Список отверстий панели

Material

Текущий материал

Синтаксис:

Material;

Тип данных:

String

Material: свойство типа String - "Текущий материал"

Result

Результат вычисления параметра

Синтаксис:

Result;

Тип данных:

Number

Result: свойство типа Number - 'Результат вычисления параметра'

Objects

Объекты скрипта

Синтаксис:

Objects;

Тип данных:

EstimateObjectList

EstimateObject

Объект сметы

Методы:

AsPanel – Получить объект как панель.

IsPanel – Проверить, является ли объект панелью.

AsButt – Получить объект как кромку.

IsButt – Проверить, является ли объект кромкой.

AsPlastic – Получить объект как пластик.

IsPlastic – Проверить, является ли объект пластиком.

AsExtrusionBody – Получить объект как профиль.

IsExtrusionBody – Проверить, является ли объект профилем.

AsTrajectoryBody – Получить объект как тело по траектории.

IsTrajectoryBody – Проверить, является ли объект телом по траектории.

Методы

AsPanel

Получить объект как панель.

Синтаксис:

AsPanel();

Возвращаемое значение:

Panel

IsPanel

Проверить, является ли объект панелью.

Синтаксис:

IsLine();

Возвращаемое значение:

Boolean

AsButt

Получить объект как кромку.

Синтаксис:

AsButt();

Возвращаемое значение:

PanelButt

IsButt

Проверить, является ли объект кромкой.

Синтаксис:

IsButt();

Возвращаемое значение:

Boolean

AsPlastic

Получить объект как пластик.

Синтаксис:

AsPlastic();

Возвращаемое значение:

PanelPlastic

IsPlastic

Проверить, является ли объект пластиком.

Синтаксис:

IsPlastic();

Возвращаемое значение:

Boolean

AsExtrusionBody

Получить объект как профиль.

Синтаксис:

AsExtrusionBody();

Возвращаемое значение:

Extrusion

IsExtrusionBody

Проверить, является ли объект профилем.

Синтаксис:

IsExtrusionBody();

Возвращаемое значение:

Boolean

AsTrajectoryBody

Получить объект как тело по траектории.

Синтаксис:

AsTrajectoryBody();

Возвращаемое значение:

Trajectory

IsTrajectoryBody

Проверить, является ли объект телом по траектории.

Синтаксис:

IsTrajectoryBody();

Возвращаемое значение:

Boolean

EstimateObjectList

Список объектов сметы

Свойства:

Count – Количество объектов.

Items – Массив объектов.

Свойства

Count

Количество объектов.

Синтаксис:

Count;

Тип данных:

Number

Items

Массив объектов.

Синтаксис:

Items;

Тип данных:

EstimateObject

PanelHoles

Список отверстий панели

Свойства:

Count – Количество отверстий.

Holes – Массив объектов.

Свойства

Count

Количество отверстий.

Синтаксис:

Count;

Тип данных:

Number

Holes

Массив объектов.

Синтаксис:

Holes;

Тип данных:

PanelHole

PanelHole

Отверстие панели

Свойства:

Depth – Глубина отверстия.

Diameter – Диаметр отверстия.

Свойства

Depth

Глубина отверстия.

Синтаксис:

Depth;

Тип данных:

Number

Diameter

Диаметр отверстия.

Синтаксис:

Diameter;

Тип данных:

Number

Мастер построения дверей

DoorsMaker

Мастер построения дверей

Методы:

Silent – Построить двери без открытия формы выбора шаблонов.

ShowErrors – Показать список ошибок после построения.

Save – Сохранить параметры установки дверей в файл.

Load – Загрузить параметры установки дверей из файла.

Setup – Установить двери в секцию.

Методы

Silent

Построить двери без открытия формы выбора шаблонов.

Синтаксис:

Silent();

Входные параметры:

Boolean

ShowErrors

Показать список ошибок после построения.

Синтаксис:

ShowErrors();

Входные параметры:

Boolean

Save

Сохранить параметры установки дверей в файл.

Синтаксис:

Save(filename);

Входные параметры:

filename (string)

Load

Загрузить параметры установки дверей из файла.

Синтаксис:

Load(filename);

Входные параметры:

filename (string)

Setup

Установить двери в секцию.

Синтаксис:

Setup(LeftObject,RightObject,TopObject,BottomObject);

Входные параметры:

LeftObject – левая граница, Panel или Edge

RightObject – правая граница, Panel или Edge

TopObject – верхняя граница, Panel или Edge

BottomObject – нижняя граница, Panel или Edge

Мастер построения ящиков

BoxesMaker

Мастер построения ящиков

Методы:

ShowErrors – Показать список ошибок после построения.

Save – Сохранить параметры установки дверей в файл.

Load – Загрузить параметры установки дверей из файла.

Setup – Установить двери в секцию.

Методы

ShowErrors

Показать список ошибок после построения.

Синтаксис:

ShowErrors();

Входные параметры:

Boolean

Save

Сохранить параметры установки ящиков в файл.

Синтаксис:

Save(filename);

Входные параметры:

filename (string)

Load

Загрузить параметры установки ящиков из файла.

Синтаксис:

Load(filename);

Входные параметры:

filename (string)

Setup

Установить ящики в секцию.

Синтаксис:

Setup(LeftObject,RightObject,TopObject,BottomObject);

Входные параметры:

LeftObject – левая граница, Panel

RightObject – правая граница, Panel

TopObject – верхняя граница, Panel или Edge

BottomObject – нижняя граница, Panel или Edge

БАЗИС-Салон

TovarItems

Информация о товаре

Свойства:

TovarName – наименование модели (товара).

TovarModel – список всех объектов модели (товара).

Price – цена товара.

IsNotStandart – стандартное или не стандартное изделие.

KodTovID – дополнительный код товара.

Metka – показывать метку и размеры.

Discount – скидка на товар.

Markup – наценка на товар.

Методы:

FindByName – поиск элемента по имени.

Свойства

TovarName

Наименование модели (товара).

Синтаксис:

TovarName;

Тип данных:

string

TovarModel

Модель 3D (список всех объектов модели).

Синтаксис:

TovarModel;

Тип данных:

List3D

Price

Цена товара.

Синтаксис:

Price;

Тип данных:

double

IsNotStandart

Стандартное или нестандартное изделие.

Синтаксис:

IsNotStandart;

Тип данных:

boolean

KodTovID

Дополнительный код товара.

Синтаксис:

KodTovID;

Тип данных:

string

Metka

Показывать метку и размеры.

Синтаксис:

Metka;

Тип данных:

boolean

Discount

Скидка на товар.

Синтаксис:

Discount;

Тип данных:

number

Примечание:

Целочисленное значение.

Markup

Наценка на товар.

Синтаксис:

Markup;

Тип данных:

number

Примечание:

Целочисленное значение.

Методы

FindByName

Поиск элемента по имени.

Синтаксис:

FindByName(name, CaseSensitive);

Входные параметры:

name – имя элемента, string

CaseSensitive – учет регистра при поиске, boolean (true – учитывать, false – нет)

Возвращаемое значение:

TScItemTovar

TScItemTovar

Свойства элемента товара

Свойства изменяемые:

Article – артикул элемента.

Name – наименование элемента.

Count – Количество элементов товара.

Material – текущий материал элемента.

Price – цена элемента.

Свойства только для чтения:

GroupMaterial – артикул элемента.

TypeElement – тип элемента.

ObjList – список 3D объектов элемента.

Свойства

Article

артикул элемента.

Синтаксис:

Article;

Тип данных:

string

Name

Наименование элемента.

Синтаксис:

Name;

Тип данных:

string

Count

Количество элементов товара.

Синтаксис:

Count;

Тип данных:

double

Material

Текущий материал элемента.

Синтаксис:

Material;

Тип данных:

string

Price

Цена элемента.

Синтаксис:

Price;

Тип данных:

double

GroupMaterial

Группа, к которой принадлежит материал.

Синтаксис:

GroupMaterial;

Тип данных:

string

Примечание:

Свойство только для чтения

TypeElement

Тип элемента.

Синтаксис:

TypeElement;

Тип данных:

string

Примечание:

Свойство только для чтения

Примечание:

Свойство только для чтения

ObjList

Список 3D объектов элемента.

Синтаксис:

ObjList;

Тип данных:

List3D

Примечание:

Свойство только для чтения

SalonUtils

Внутренние функции БАЗИС-Салон

Свойства:

PathAttachments – путь к папке с прикрепленными файлами.

SalonId – идентификатор Салона.

Методы:

GetFullPathAttachment – получить путь к файлу.

Свойства

PathAttachments

Путь к папке с прикрепленными файлами.

Синтаксис:

PathAttachments;

Тип данных:

string

Примечание:

Свойство только для чтения

SalonId

Идентификатор Салона.

Синтаксис:

SalonId;

Тип данных:

string

Методы

GetFullPathAttachment

Поиск элемента по имени.

Синтаксис:

FindByName(name, CaseSensitive);

Входные параметры:

fileName – имя файла, string

Возвращаемое значение:

Если файл будет находится в папке с прикрепленными файлами, то вернется полный путь файла, иначе только его имя.

UserProperties

Пользовательские свойства

Свойства:

Value – значение по имени свойства.

Name – получить имя свойства по индексу.

Count – количество свойств.

Методы:

Remove – удалить свойство по имени.

Свойства

Value

Значение по имени свойства.

Синтаксис:

Value;

Тип данных:

Array

Name

Получить имя свойства по индексу.

Синтаксис:

Name;

Тип данных:

Array.<String>

Count

Количество свойств.

Синтаксис:

Count;

Тип данных:

Number

Методы

Remove

Удалить свойство по имени.

Синтаксис:

Remove(name);

Входные параметры:

name – имя свойства, string

Приложение I. Создание экранной формы в визуальном режиме

Экранная форма может быть создана в визуальном режиме. При этом автоматически генерируется текст скрипта, который обеспечит при запуске создание этой формы. Чтобы выполнить построения, необходимо вызвать команду **Скрипты** – **Редактор скриптов**. На экране появится диалог **Редактор скриптов** (рис. 1).

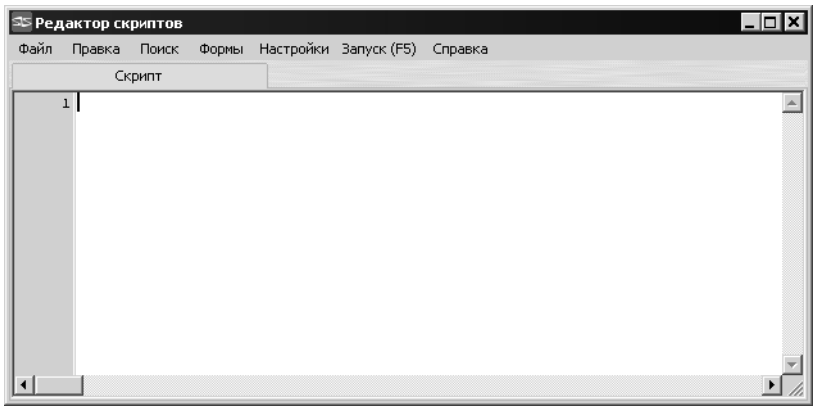


Рис. 1.

В этом диалоге вызовите команду **Формы** – **Добавить форму**. На экране появится диалог **Редактор форм** (рис. 2).

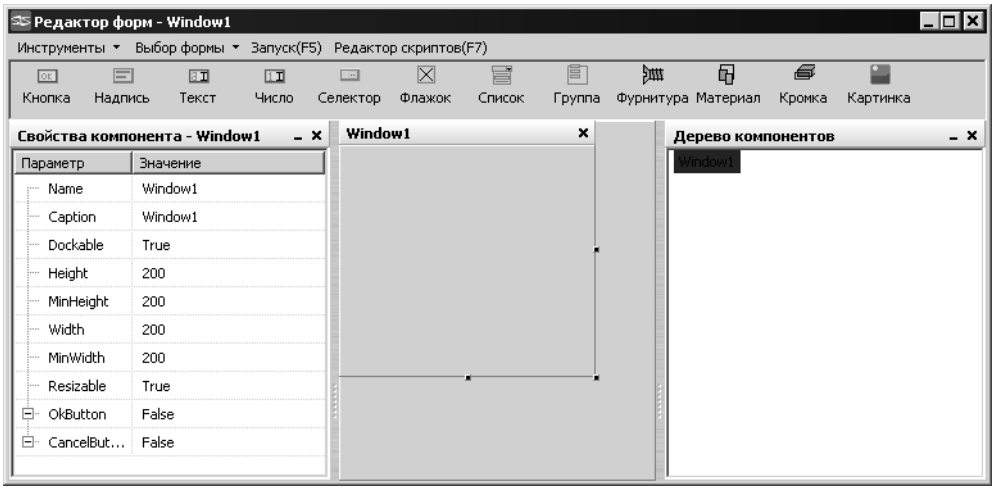


Рис. 2.

В этом диалоге автоматически создается экранная форма, имеющая умолчательные параметры. Свойства этого компонента, на данный момент единственного, показаны на панели **Свойства компонента**. Панель **Дерево компонентов** тоже содержит обозначение единственного компонента, окна формы. Наличие этих панелей в окне **Редактор**

форм определяется командами меню **Инструменты**. Геометрические размеры компонента можно изменить как используя курсор мыши, так и задавая значения его свойств. Элементы управления, расположенные на панели **Свойства компонента**, позволяют изменять и другие свойства. Чтобы проверить функционирования компонента, следует вызвать команду **Запуск**. Созданная форма появится на экране. Чтобы продолжить ее редактирование, форму необходимо закрыть.

Результаты визуального проектирования приводят к автоматической генерации текста скрипта. Он формируется в диалоге **Редактор скриптов**, который вовремя визуального проектирования остается открытым. Команда **Редактор скриптов** позволяет сделать этот диалог активным.

Панель компонентов содержит обозначения элементов управления, которые можно использовать при создании экранной формы. Компоненты *Кнопка, Надпись, Текст, Число, Селектор, Флажок, Список и Картинка* являются стандартными элементами управления. Компоненты *Фурнитура, Материал, Кромка* являются специфическими для системы БАЗИС-Мебельщик. Они используются для выбора соответствующих объектов из Базы данных материалов. Компонент *Группа* обеспечивает возможность создания иерархической структуры компонентов экранной формы. Чтобы поместить компонент на форме, следует использовать стандартную технологию Drag-and-Drop. После того, как компонент будет расположен в окне, он появится в структуре объектов формы на панели **Дерево компонентов**.

В редакторе может одновременно быть открыто для редактирования несколько форм. Команды меню **Выбор формы** позволяют выбрать нужную из них для редактирования. Для создания новой формы, вызовите команду **Добавить форму** этого меню.

Чтобы завершить визуальное проектирование экранных форм, закройте окно **Редактор форм**.

A
Action *49*
Action3D *70*
ArrangePositions *79*
AsyncExec *76*
BeginOrtho3 *76*
BlinkHint *73*
Cancel *76*
ChooseMaterial *79*
Commit *75*
Continue *75*
CursorToClosestLine *77*
CursorToClosestPoint *77*
CursorToMiddleOfLine *77*
EndOrtho3 *77*
ErrorHint *73*
Find3DPoint *77*
Find3DPointXZPlane *77*
Finish *75*
Get3Dobject *77*
Hint *73*
Interactive *71*
LoadModel *78*
ModelFilename *75*
MouseX *71*
MouseY *72*
NewFragment *78*
NewFurniture *78*
NewModel *78*
OnClick *73*
OnFinish *74*
OnMove *74*
OnRayTraceFinished *75*
OnStart *74*
Pos3 *72*
Properties *74*
RayTraceScene *78*
ReplaceFurniture *79*
Revert *79*
RightDir *72*
SaveModel *78*
ShowEdges *73*
ShowPoints *73*
UpDir *72*
ViewDir *72*
ActiveMaterial *49*
AddAssembly *56*
AddBlock *56*
AddCopy *57*
AddDraftBlock *56*
AddExtrusion *55*
AddFrontPanel *53*
AddHorizPanel *54*
AddPanel *53*
AddTrajectory *55*
AddVertPanel *54*
alert *50*
AnimationType *49*
Arc2D *165*
ArcDir *166*
Center *166*
Pos1 *166*
Pos2 *166*
Assembly *127*
AnimType *127*
Axis_X *44*
Axis_Y *44*
Axis_Z *45*
AxisX *44*
AxisY *44*
AxisZ *44*

B
BeginBlock *58*
Block *126*
IsFastener *126*
BoxesMaker *177*
Load *178*
Save *178*
Setup *178*
ShowErrors *177*

C
Circle2D *167*
Center *167*

CirRadius *167*
Dir *167*
Clear *84*
confirm *50*
Contour2D *135*
AddArc *140*
AddArc3 *140*
AddCircle *140*
AddEquidistant *144*
AddEquidistantRecursive *141*
Addition *141*
AddLine *140*
AddRectangle *139*
AddRoundRect *139*
Clear *138*
Count *137*
Elastic *143*
Facet *142*
FacetEx *142*
Find *143*
Fit *143*
Height *138*
IsClosedContour *145*
IsContourRectangle *145*
IsInContour *145*
IsPointInside *144*
Load *144*
Max *138*
Min *138*
Move *139*
Objects *137*
OrderContours *144*
Rotate *139*
Rounding *142*
RoundingEx *141*
Subtraction *141*
Symmetry *143*
Width *138*
Count *69*

D
DatumMode *47*
DeleteNewObjects *57*
DeleteObject *57*
DoorsMaker *176*
Load *177*
Save *176*
Setup *177*
ShowErrors *176*
Silent *176*

E
Edge3 *105*
First *105*
GFirst *105*
GLast *105*
Last *105*
Elem2D *162*
AsArc *164*
AsCircle *163*
AsEllipse *164*
AsLine *163*
AsList *165*
ElType *163*
IsArc *164*
IsCircle *163*
IsEllipse *164*
IsLine *163*
IsList *164*
ObjLength *165*
ElementType *48*
Ellipse2D *167*
Center *168*
Dir *168*
MajorAxisAngle *168*
MajorRadius *168*
MinorRadius *168*
EndBlock *58*
ErrorType *47*
EstimateObject *172*
AsButt *173*
AsExtrusionBody *173*
AsPanel *172*
AsPlastic *173*

AsTrajectoryBody *174*
IsButt *173*
IsExtrusionBody *174*
IsPanel *172*
IsPlastic *173*
IsTrajectoryBody *174*
EstimateObjectList *174*
Count *174*
Items *174*
ExtractMatCode *62*
ExtractMatName *61*
Extrusion *124*
Contour *124*
MaterialName *124*
Thickness *124*

F
FormatMatName *61*
FurnArticle *169*
DatumMode *170*
Name *169*
NameWithCode *170*
OrderCode *169*
OrderName *169*
FurniturePosition *48*
FurnMaterial *102*
Make *103*
Name *103*
Thickness *103*
Width *103*
FurnPositionMode *48*
FurnSchemeType *46*

G
Geometry2D *62*
Area *63*
Compare *62*
Distance *63*
Intersect *62*
GetEdge *52*
GetObject *52*
GetPanel *52*
GetPoint *52*

H
Holes *171*

I
ImportExport
SVG *159*
ImportExportSVG *157*
CurveQuality *158*
GroupElems *158*
Load *158*
Save *158*
InButtMaterial *150*
Allowance *151*
ClipPanel *151*
Name *150*
Overhung *151*
Sign *151*
Thickness *151*
Width *151*
InButton *146*
Caption *147*
Combo *147*
NewSubMenu *147*
InControl *145*
Enabled *146*
Hint *146*
id *145*
OnChange *146*
Visible *146*
InfFurniture *159*
Choose *162*
DatumMode *160*
DatumModeFilter *159*
DecodeFromString *161*
EncodeToString *161*
Make *161*
Mount *160*
Mount1 *160*
MountBox *161*
MountScheme *160*
InFloat *147*
Fixed *148*

ReadOnly *148*
Value *147*
InFurniture *152*
DatumMode *152*
DatumModeFilter *152*
Mount *152*
Mount1 *153*
MountBox *153*
MountScheme *153*
InMateria
Apply *150*
Name *149*
SetActive *150*
Thickness *149*
Width *149*
InMaterial *149*
InNumber *148*
Fixed *149*
ReadOnly *148*
Value *148*
InspectorError *156*
ErrorMessage *157*
ErrorObjects *157*
ErrorObjectsCount *157*
ErrorType *156*
ObjectsNames *157*
InspectorOptions *154*
FastIncorrectAnalyze *155*
FastIntersectionAnalyze *155*
MatNotExistsAnalyze *156*
MatOutOfStockAnalyze *156*
ObjIntersectionAnalyze *155*
PanelNotFixedAnalyze *155*
PanelTooLargeAnalyze *155*
PlasticTooLargeAnalyze *156*
Items *82*

L

Line2D *165*
Pos1 *165*
Pos2 *165*
List3D *116*
Count *117*
ElasticResize *118*
FurnType *118*
IsElastic *117*
Load *118*
Objects *117*
SalonType *118*

M

Material *171*
MaxValue *91*
MinValue *91*
Model *49*
Model3D *69*
Objects *70*
Selected *69*
SelectionCount *69*
Selections *69*
ModelInspector *153*
ErrorList *154*
Options *154*
Run *154*

N

NewButtMaterialInput *60*
NewButtonInput *58*
NewCOMObject *43*
NewContour *43*
NewFloatInput *59*
NewForm *44*
NewFurnitureInput *61*
newlink ScriptForm
ShowModal *102*
NewMaterialInput *60*
NewNumberInput *59*
NewPoint *43*
NewVector *42*

O

Object3 *106*
ArtPos *107*
AsList *108*
AsPanel *109*

Build *116*
GabMax *110*
GabMin *110*
GMax *109*
GMin *110*
GSize *110*
IsOwner *116*
List *108*
Name *107*
NToGlobal *116*
NToObject *116*
Orient *113*
OrientGCS *114*
Owner *108*
Position *109*
PositionX *109*
PositionY *109*
PositionZ *109*
Reflect *115*
Rotate *112*
RotateGCS *112*
RotateX *113*
RotateY *113*
RotateZ *113*
Selected *108*
SetDefaultTransform *111*
ToGlobal *115*
ToObject *115*
Translate *112*
TranslateGCS *112*
UserPropCount *111*
UserProperty *111*
UserPropertyName *111*
Visible *108*
Objects *172*
OpenFurniture *50*
OrientCamera *61*

P

p3dBottom *171*
p3dFront *170*
p3dIsometric *171*
p3dLeft *170*
p3dRight *170*
p3dTop *170*
Pane
Butts *120*
Panel *119*, *171*
AddButt *122*
AddCut *123*
AddPlastic *123*
Contour *119*
ContourHeight *120*
ContourWidth *120*
Cuts *120*
IsButtVisible *123*
MaterialName *121*
MaterialWidth *121*
Plastics *120*
Thickness *121*
PanelButt *128*
Allowance *130*
ClipPanel *130*
CutIndex *130*
ElemIndex *129*
Material *129*
Overhung *130*
Profile *131*
Sign *129*
Thickness *129*
Width *130*
PanelButts *127*
Add *128*
Butts *128*
Count *127*
PanelCut *134*
Contour *135*
Name *134*
Sign *134*
Trajectory *135*
PanelCuts *133*
Add *134*
Count *133*

Cuts *134*
PanelHole *175*
Depth *175*
Diameter *176*
PanelHoles *175*
Count *175*
Holes *175*
PanelPlastic *132*
Material *132*
TextureOrientation *133*
Thickness *133*
PanelPlastics *131*
Add *132*
Count *131*
PanelSideEnum *45*
Plastics *132*
Point *104*
x *104*
y *105*
prompt *49*

R

Result *171*
RootProperties *162*
NewFurnitureValue *162*

S

SalonAnimationType *45*
SalonUtils *183*
GetFullPathAttachment *183*
PathAttachments *183*
SalonId *183*
SchemeFurnPos *46*
ScriptBooleanProperty *90*
ScriptButtonProperty *92*
OnClick *92*
ScriptButtProperty *93*
SetActive *93*
Thickness *93*
Width *93*
ScriptColorProperty *94*
Value *94*
ScriptForm *97*
CancelButton *100*
CancelButtonCaption *100*
Caption *98*
Close *102*
Dockable *101*
Height *99*
Left *100*
MinHeight *99*
MinWidth *99*
OKButton *100*
OKButtonCaption *100*
OnCancelButtonClick *102*
OnClose *101*
OnShow *101*
Properties *98*
Resizable *101*
Show *102*
Top *100*
Visible *99*
Width *99*
ScriptForm_OnOkButtonClick *101*
ScriptFurnitureProperty *94*
Value *94*
ScriptGroupProperty *89*
Image *89*
MaxHeight *89*
ScriptBooleanProperty *90*
ScriptNumberProperty *90*
Scrollable *89*
Value *90*
ScriptMaterialProperty *92*
SetActive *93*
Width *92*
ScriptMenu *94*
Clear *96*
Count *95*
Items *95*
Load *96*
Name *95*
NewBool *97*
NewButton *97*

NewGroup *96*
NewNumber *97*
NewString *97*
OnChange *95*
Save *96*
Store *95*
ScriptProperty *79*
BackColor *83*
ChildrenEnabled *81*
Count *82*
DropDownMenu *83*
Enabled *81*
Expanded *82*
Load *85*
Name *81*
NameEditable *83*
NewBool *86*
NewButt *87*
NewButton *87*
NewCOMObject *84*
NewFurniture *88*
NewGroup *85*
NewImage *85*
NewMaterial *87*
NewNumber *86*
NewSeparator *88*
NewString *86*
OnActivate *83*
OnChange *81*
OnDeactivate *82*
OnValueChange *84*
OnValueValidate *82*
PopupMenu *83*
Save *84*
Tag *84*
Validate *88*
ValueValid *83*
Visible *81*
ScriptStringProperty *90*
SelectAll *51*
SetCamera *51*
StartEditing *58*
System *63*
apiVersion *68*
askFileNameSave *67*
askFolder *67*
askReadTextFile *65*
askWriteTextFile *65*
developerApiVersion *68*
exec *66*
fileExists *64*
getFileName *67*
getFileNameWithoutExtension *67*
include *64*
log *64*
readTextFile *65*
secureExec *65*
writeTextFile *64*
system *48*

T

TextureDir *46*
TovarItems *178*
Discount *180*
FindByName *180*
IsNotStandart *179*
KodTovID *179*
Markup *180*
Metka *180*
Price *179*
TovarModel *179*
TovarName *179*
TProjection3dEnum *45*
Trajectory *125*
Contour2D *125*
MaterialName *126*
Trajectory2D *125*
TScItemTovar *181*
Article *181*
Count *181*
GroupMaterial *182*
Material *182*
Name *181*

ObjList *182*
Price *182*
TypeElement *182*

U
UnSelectAll *51*
UserProperties *184*
Count *184*
Name *184*
Remove *185*
Value *184*

V
Value *90*, *91*, *92*
ValueStep *91*
Vector *103*
x *104*
y *104*
z *104*
ViewAll *51*

W
windowposition *46*

X
x *104*, *105*